Introduction to SQL

(Unit-4)

Overview of SQL Query Language

IBM developed the original version of SQL, originally called Sequel, as part of the System R project in the early 1970s. The Sequel language has evolved since then, and its name has changed to SQL (Structured Query Language). Many products now support the SQL language.

SQL (Structured Query Language) is a programming language designed for managing and manipulating relational databases. It is widely used in database management systems (DBMS) to create, retrieve, update, and delete data from databases. SQL provides a standardized syntax and set of commands that allow users to interact with the database in a declarative manner. SQL is a declarative language, which means that you tell the database what you want, and the database figures out how to do it. This is in contrast to procedural languages, where you tell the database how to do something. Structured Query Language (SQL) is a domain-specific language designed for managing data held in a relational database management system (RDBMS). It is particularly useful in handling structured data, i.e. data incorporating relations among entities and variables.

SQL language has several parts:

- Data-definition language (DDL). The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- Data-manipulation language (DML). The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- Integrity. The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- View definition. The SQL DDL includes commands for defining views.
- Transaction control. SQL includes commands for specifying the beginning and ending of transactions.
- Embedded SQL and dynamic SQL. Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, and Java.
- Authorization. The SQL DDL includes commands for specifying access rights to relations and views.

- Data Control: SQL provides statements for managing user access and privileges. The GRANT statement is used to grant permissions to users, and the REVOKE statement is used to revoke or remove permissions.
- Data Integrity: SQL supports the enforcement of data integrity rules through constraints. You can define constraints such as primary key, foreign key, unique, and check constraints to ensure the validity and consistency of data.
- Data Aggregation: SQL includes aggregate functions such as SUM, AVG, COUNT, MAX, and MIN, which allow you to perform calculations on groups of rows and retrieve summarized information.
- Joins and Relationships: SQL supports various types of joins (e.g., INNER JOIN, LEFT JOIN, RIGHT JOIN) to combine data from multiple tables based on related columns. This enables you to establish relationships between tables and retrieve data from related entities.

SQL Data Definition

SQL Data Definition Language (DDL) is a subset of SQL that is used to define and manage the structure of a database. It includes statements that allow you to create, modify, and delete database objects such as tables, views, indexes, and constraints. The SQL DDL allows specification of not only a set of relations, but also information about each relation, including:

- It involves defining the schema or blueprint of the database, specifying the organization and characteristics of the data.
- The schema for each relation.
- The types of values associated with each attribute.
- The integrity constraints.
- The set of indices to be maintained for each relation.
- The security and authorization information for each relation.
- The physical storage structure of each relation on disk.

1. Basic Types

The SQL standard supports a variety of built-in types, including:

- char(n): A fixed-length character string with user-specified length n. The full form, character, can be used instead.
- varchar(n): A variable-length character string with user-specified maximum length n. The full form, character varying, is equivalent.

- int: An integer (a finite subset of the integers that ismachine dependent). The full form, integer, is equivalent.
- smallint: A small integer (a machine-dependent subset of the integer type).
- numeric(p, d):A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point. Thus, numeric(3,1) allows 44.5 to be stored exactly, but neither 444.5 or 0.32 can be stored exactly in a field of this type.
- real, double precision: Floating-point and double-precision floating-point numbers with machine-dependent precision.
- float(n): A floating-point number, with precision of at least n digits.
- Each type may include a special value called the null value.

2. Basic Schema Definition

A schema is a blueprint of a database. It defines the tables, columns, and data types that are used to store data. A schema is used to create a database and to ensure that the data in the database is consistent.

```
create table instructor (ID varchar (5),
name varchar (20) not null,
dept name varchar (20),
salary numeric (8,2),
primary key (ID),
foreign key (dept name) references department);
```

Here are the basic components of a schema definition:

- Tables: Tables are the primary database objects that store and organize data.
- Columns: Columns define the individual data elements or attributes within a table
- Primary Key: A primary key is a column or combination of columns that uniquely identifies each record in a table.
- Foreign Key: A foreign key is a column or set of columns in a table that establishes a link or relationship with the primary key of another table.
- Constraints: Constraints define rules or conditions that the data in the database must adhere to. Common types of constraints include unique constraints, which ensure the uniqueness of values in a column or set of columns, and referential integrity constraints, which enforce the validity of relationships between tables.

- Indexes: Indexes are data structures that improve the performance of data retrieval operations by allowing faster access to specific data.
- Views: Views are virtual tables that are derived from one or more base tables.

3. **Drop/Alter**

- `ALTER TABLE table_name ADD column_name datatype;`
- `ALTER TABLE table_name DROP COLUMN column_name;`
- `ALTER TABLE table_name ADD CONSTRAINT constraint_name PRIMARY KEY (col1);`
- `ALTER TABLE orders ADD CONSTRAINT users_fk FOREIGN KEY (user_id) REFERENCES users (user_id);`
- `ALTER TABLE table DROP CONSTRAINT constraint_name`
- `ALTER TABLE table_name RENAME COLUMN old_column_name TO new_column_name;`
- `ALTER TABLE old_table RENAME new_table;`
- `DROP TABLE table_name;`
- `DROP TABLE table_name;`

Basic Structure of SQL Queries

The basic structure of an SQL query consists of three clauses: select, from, and where. The query takes as its input the relations listed in the from clause, operates on them as specified in the where and select clauses, and then produces a relation as the result.

1. **Queries on a Single Relation**

Queries on a single relation involve retrieving, filtering, and manipulating data from a single table in a database. Here are some common types of queries that can be performed on a single relation:

- `Select * from table_name`
- `Select column1, column2 from table_name;`
- `SELECT * FROM offices WHERE country = 'USA';`
- `SELECT * FROM offices WHERE office_code > 5;`
- `select name from instructor where dept name = 'Comp. Sci.' and salary > 70000;`

- `select name from instructor where salary between 90000 and 100000;`
- `SELECT COUNT(*) FROM orders;`
- `SELECT * FROM products ORDER BY price DESC;`

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=

2. **Queries on multiple relations**

   Queries on multiple relations involve retrieving, combining, and manipulating data from multiple tables in a database. These queries allow you to extract meaningful information by joining and filtering data across related tables. Here are some common types of queries that can be performed on multiple relations:

For Example:- select *name, course id* **from** *instructor, teaches* **where** *instructor.ID= teaches.ID;*

3. **The Natural Join**

The natural join is a type of join operation in SQL that combines rows from two or more tables based on columns with the same name and data type.

SELECT columns FROM table1 NATURAL JOIN table2;

Additional Basic Operations

1. **The Rename Operation**

   The rename operation in SQL allows you to change the name of a relation or attribute in a query result. It is typically used to provide more meaningful or concise names to the columns or tables in the output of a query.

   ```
   SELECT expression AS new_name FROM table_name;
   ```

In this syntax, *expression* represents the column or expression you want to select from the table, and *new_name* represents the new name you want to assign to that column or expression in the query result.

```
select distinct T.name from instructor as T, instructor as
S where T.salary > S.salary and S.dept name = 'Biology';
```

2. **String Operations**

String operations in SQL allow you to manipulate and perform operations on string values stored in the database. SQL provides several built-in string functions that can be used to manipulate and process strings.

SQL also permits a variety of functions on character strings, such as **concatenating** (using "||"), extracting substrings, finding the **length** of strings, converting strings to uppercase (using the function **upper(s)** where s is a string) and lowercase (using the function **lower(s)),** removing spaces at the end of the string (using **trim(s))** and so on.

SQL provides the LIKE operator and the REGEXP functions for pattern matching in strings. These can be used to search for specific patterns or perform regular expression matching on string values.

```
SELECT product_name FROM products WHERE product_name LIKE
'T%';
```

This query retrieves all product names from the products table that start with the letter 'T'.

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

3. **Attribute Specification in Select Clause**

   The asterisk symbol " * " can be usedin the select clause to denote "all attributes."

4. **Ordering the Display of Tuples**

   SQL offers the user some control over the order in which tuples in a relation are displayed. The order by clause causes the tuples in the result of a query to appear in sorted order

   ```
   select * from instructor order by salary desc.
   ```

   By default, the order by clause lists items in ascending order. To specify the sort order, we may specify **desc** for descending order or **asc** for ascending order.

5. **Where Clause Predicates**

   In SQL, the WHERE clause is used to specify conditions for filtering rows in a query. Predicates are logical conditions that are used in the WHERE clause to define the criteria for selecting rows from a table.

   ```
   select name from instructor where salary between 90000 and
   100000;
   ```

   ```
   Or,
   ```

   ```
   select name from instructor where salary <= 100000 and
   salary >= 90000;
   ```

   ```
   select name, course id from instructor, teaches where
   instructor.ID= teaches.ID and dept name = 'Biology';
   ```

Set Operations

The SQL operations union, intersect, and except operate on relations and correspond to the mathematical set-theory operations ∪, ∩, and −. We shall now construct queries involving the union, intersect, and except operations over two sets. It's important to note that for set operations to work, the columns in the SELECT statements must have compatible data types and be in the same order.

1. UNION: The UNION operator combines the results of two SELECT statements into a single result set, removing duplicate rows. The syntax is as follows:

   ```
   SELECT column1, column2, ... FROM table1
   UNION
   SELECT column1, column2, ... FROM table2;
   ```

2. INTERSECT: The INTERSECT operator returns the common rows between the results of two SELECT statements, removing duplicate rows. The syntax is as follows:

```
SELECT column1, column2, ... FROM table1
INTERSECT
SELECT column1, column2, ... FROM table2;
```

3. Except (Or Minus) : The EXCEPT operator returns the rows that are present in the result of the first SELECT statement but not in the result of the second SELECT statement. The syntax is as follows:

```
SELECT column1, column2, ... FROM table1
EXCEPT
SELECT column1, column2, ... FROM table2;
```

Null Values

In SQL, a null value represents the absence of a value or an unknown value. It is different from zero or an empty string, as it indicates the lack of a value altogether. A null value is represented by the keyword NULL.

```
select name from instructor where salary is null;
```

This query retrieves the names of instructors who have a null value for the "salary" column. The result will include all rows where the "salary" value is null, and it will display only the "name" column from the "instructor" table.

Aggregate Functions

Aggregate functions in SQL are used to perform calculations on a set of rows and return a single result. Here are some commonly used aggregate functions:

- COUNT: Returns the number of rows in a specified column or table.
- SUM: Calculates the sum of values in a specified column.
- AVG: Calculates the average of values in a specified column.
- MAX: Retrieves the maximum value in a specified column.
- MIN: Retrieves the minimum value in a specified column.

These functions can be used in combination with the SELECT statement to retrieve aggregated data from a table.

1. **Basic Aggregation**

```
select avg (salary) from instructor where dept name= 'Comp.
Sci.';
```

The result of this query is a relation with a single attribute, containing a single tuple with a numerical value corresponding to the average salary of instructors in the Computer Science department.

```
SELECT COUNT(*) AS total_records,
       SUM(sales) AS total_sales,
       AVG(price) AS average_price,
       MAX(quantity) AS max_quantity,
       MIN(order_date) AS earliest_order_date
FROM orders;
```

2. **Aggregation with Grouping**

There are circumstances where we would like to apply the aggregate function not only to a single set of tuples, but also to a group of sets of tuples; we specify this wish in SQL using the group by clause. The attribute or attributes given in the group by clause are used to form groups.

```
select dept name, avg (salary) as avg salary from
instructor group by dept name;
```

3. **The Having Clause**

The HAVING clause in SQL is used to filter the result set based on a condition applied to aggregated values. It is similar to the WHERE clause, but it operates on the result of the GROUP BY clause. The HAVING clause is typically used in conjunction with the GROUP BY clause to perform filtering on grouped data.

```
SELECT column1, column2, ... FROM table GROUP BY column1,
column2, ...
HAVING condition;
```

4. **Aggregation with Null and Boolean Values**

In general, aggregate functions treat nulls according to the following rule: All aggregate functions except count (*) ignore null values in their input collection. As a result of null values being ignored, the collection of values may be empty. The count of an empty collection is defined to be 0, and all other aggregate operations return a value of null when applied on an empty collection.

The effect of null values on some of the more complicated SQL constructs can be subtle. A Boolean data type that can take values true, false, and unknown, was introduced in SQL:1999. The aggregate functions some and every, which mean exactly what you would intuitively expect, can be applied on a collection of Boolean values.

Nested Subqueries

Nested subqueries, also known as subquery within a subquery or nested query, refer to the concept of using a subquery (a query inside another query) within the context of another query. The result of the inner subquery is used in the outer query to perform further filtering or calculations.

Nested subqueries allow you to break down complex problems into smaller, more manageable parts and tackle them step by step. The result of the inner subquery is treated as a temporary table or dataset that can be used in the outer query just like any other table or dataset.

1. **Set Comparison**
   Nested subqueries can be used to perform set comparisons in SQL. Set comparisons involve comparing the results of two or more queries to determine common or distinct values. It's important to note that the efficiency and performance of set comparison queries using nested subqueries depend on factors such as the size of the result sets, available indexes, and query optimization.

   Set comparison with SOME and ALL is used to compare a single value with a set of values in a subquery. These operators are often used in combination with other comparison operators such as <, >, =, etc.

   SOME (also known as ANY): It returns true if any of the values in the subquery satisfy the comparison condition.

```
select name from instructor where salary > some (select
salary from instructor where dept name = 'Biology');
```

   ALL: It returns true if all the values in the subquery satisfy the comparison condition.

```
select name from instructor where salary > all (select
salary from instructor where dept name = 'Biology');
```

2. **Test for Empty Relations**

   To test for empty relations (tables) in SQL, you can use the SELECT statement with the appropriate condition. Here are a few methods to achieve this:

   - Using the NOT EXISTS clause:

     ```
     SELECT * FROM your_table WHERE NOT EXISTS ( SELECT 1
     FROM your_table );
     ```

     This query checks if there are no records in the your_table by using a subquery with NOT EXISTS that checks for the absence of any records.

   - Using the EXISTS clause:

     ```
     SELECT * FROM your_table WHERE EXISTS (  SELECT 1 FROM
     your_table  HAVING COUNT(*) = 0);
     ```

     This query uses the EXISTS clause along with the HAVING clause to check if the count of records is 0 in the your_table.

3. **Subqueries in the From Clause**

   Subqueries in the FROM clause are used to create a derived table, also known as an inline view or subquery factoring. A derived table is a temporary table that is created within the context of a larger query. The result of the subquery is treated as a table, and it can be used like any other table in the query.

   ```
   select dept name, avg salary
   from (select dept name, avg (salary) as avg salary
   from instructor
   group by dept name)
   where avg salary > 42000;
   ```

Scalar Subqueries

Scalar subqueries are subqueries that return a single value, which can be used as a scalar value in the outer query. They can be used in various parts of a SQL statement, such as the SELECT clause, WHERE clause, or even as a column in the FROM clause. Here's a simple example of using a scalar subquery in the SELECT clause:

```
select dept name,
(select count(*)
from instructor
where department.dept name = instructor.dept name)
as num instructors
from department;
```

Modification of the Database

We have restricted our attention until now to the extraction of information from the database. Now, we show how to add, remove, or change information with SQL.

1. **Delete**

A delete request is expressed in much the same way as a query. We can delete only whole tuples; we cannot delete values on only particular attributes. SQL expresses a deletion by

```
Delete from table_name where id=1;
```

Here, after "Where", we can use any column name to delete data but we prefer unique column so that collision will not occur

2. **Insertion**

To insert data into a relation, we either specify a tuple to be inserted or write a query whose result is a set of tuples to be inserted.

```
insert into course values ('CS-437', 'Database Systems', 'Comp. Sci.',
4);
```

We can use it with column name or without column name.

3. **Updates**

In certain situations, we may wish to change a value in a tuple without changing all values in the tuple. For this purpose, the update statement can be used. As we could for insert and delete, we can choose the tuples to be updated by using a query.

```
update instructor set salary= salary * 1.05 where id =1;
```

All above Modification of the Database can be use with subqueries, group by order by and so on.

Join Expression

A join expression is used to combine rows from two or more tables based on a related column between them. It specifies the conditions under which the tables are joined, determining how the rows from each table are matched and combined in the result set.

In SQL, the most common join expression is the ON clause, which is used with the JOIN keyword to specify the join condition. The ON clause typically consists of a comparison between columns from the joined tables.

1. **Join Conditions**
   Join conditions are used in SQL to specify the relationships between tables when performing a join operation. They define how rows from different tables should be matched and combined in the result set. The join condition typically involves comparing values between columns in the joined tables.

```
select * from student join takes on student.ID= takes.ID;

                    Or

select * from student, takes where student.ID= takes.ID;

            //Both Query gives same result
```

2. **Outer Joins**
   Outer joins are used to retrieve data from two or more tables, including unmatched rows from one or both of the tables involved in the join. It allows you to include rows that do not have matching values in the join condition.

   There are three types of outer joins:

   Left Outer Join (or Left Join):

   - Retrieves all rows from the left table and the matched rows from the right table.
   - If there are unmatched rows in the right table, NULL values are used for the columns of the right table in the result set.
   - *select * from student left outer join takes on student.ID= takes.ID;*

   Right Outer Join (or Right Join):

   - Retrieves all rows from the right table and the matched rows from the left table.
   - If there are unmatched rows in the left table, NULL values are used for the columns of the left table in the result set.
   - *select * from student right outer join takes on student.ID= takes.ID;*

Full Outer Join (or Full Join):

- Retrieves all rows from both the left and right tables.
- If there are unmatched rows in either table, NULL values are used for the columns of the respective table in the result set.
- *select * from student full outer join takes on student.ID= takes.ID;*

Views

A view is a virtual table that is derived from one or more existing tables or views. It does not store any data itself but rather presents a tailored representation of the data stored in the underlying tables. Views are defined by using SQL queries, and they can be used to simplify complex queries, provide a layer of abstraction, restrict access to certain data, or present data in a different format.

views offer flexibility, security, and simplicity in working with database data. They provide a way to customize and control data access while presenting a logical and meaningful view of the information stored in the database.

1. View Definition

A view definition is a statement in SQL that specifies the criteria and structure for creating a view in a database. It defines the columns, tables, and conditions used to retrieve data and generate the view.

```
CREATE VIEW view_name AS

SELECT column1, column2, ...

FROM table1

WHERE condition;
```

Let's break down the components of a view definition:

- CREATE VIEW: This is the keyword used to indicate that a view is being created.
- view_name: This is the name given to the view. It should be unique within the database.
- AS: This keyword is used to specify that the following SELECT statement is the source of data for the view.
- SELECT column1, column2, ...: This is the list of columns to be included in the view. You can specify specific columns or use * to include all columns from the source table(s).

- FROM table1: This is the source table from which the data will be retrieved. You can join multiple tables if needed.
- WHERE condition: This is an optional clause that allows you to specify conditions for filtering the data in the view. It is used to define the criteria for selecting specific rows.

2. Materialized Views

A materialized view in a database is a precomputed, stored representation of a query result. Unlike regular views, which are virtual and do not store data themselves, materialized views store the result of a query as a physical table-like structure. This allows for faster query execution and improved performance, especially for complex queries or queries involving large datasets.

Materialized views are particularly useful in scenarios where the underlying data is relatively static or where query performance is critical.

Transaction

A transaction consists of a sequence of query and/or update statements. The SQL standard specifies that a transaction begins implicitly when an SQL statement is executed. Transactions are typically used to group a set of database operations into a single logical unit of work. Transactions ensure the integrity and consistency of the database by providing the ability to either **commit** the changes made within the transaction or **roll back** those changes if needed. Savepoints, on the other hand, allow for finer-grained control within a transaction by providing intermediate points to which the transaction can be rolled back.

One of the following SQL statements must end the transaction:

- **Commit work** commits the current transaction; that is, it makes the updates performed by the transaction become permanent in the database. After the transaction is committed, a new transaction is automatically started.
  Commit is a statement used to permanently save the changes made within a transaction to the database. When a commit statement is executed, all the changes made within the transaction become permanent and visible to other transactions. Once committed, the changes cannot be rolled back, and they will persist even in the event of a system failure.
- **Rollback work** causes the current transaction to be rolled back; that is, it undoes all the updates performed by the SQL statements in the transaction. Thus, the database state is restored to what it was before the first statement of the transaction was executed.
  Rollback is a statement used to undo the changes made within a transaction and restore the database to its state before the transaction began. When a rollback statement is executed, all the changes made within the transaction are discarded, and the database is reverted to its previous state. Rollback is typically used to handle errors or exceptional situations within a transaction, ensuring that no partial or inconsistent changes are left behind.

Integrity Constraints

Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency. Thus, integrity constraints guard against accidental damage to the database. Integrity constraints in a database management system ensure the validity and integrity of data stored in the database.

Examples of integrity constraints are:

• An instructor name cannot be null.

• No two instructors can have the same instructor ID.

• Every department name in the course relation must have a matching department

name in the department relation.

• The budget of a department must be greater than $0.00.

1. **NOT NULL** Constraint: This constraint ensures that a column or attribute in a table cannot have a null value. It guarantees that the specified attribute must contain a valid value for every row in the table.
2. UNIQUE Constraint: The unique constraint ensures that the values in a column or a set of columns are unique across the table. It prevents duplicate values from being inserted into the specified column(s).
3. CHECK Constraint: The check constraint is used to define a condition that must be satisfied for the data in a column. It allows you to enforce specific rules or conditions on the values allowed in a column. For example, you can use a check constraint to ensure that a column named "age" only contains positive values.
4. REFERENTIAL INTEGRITY Constraint: The referential integrity constraint is used to establish and enforce relationships between tables. It ensures that values in a foreign key column of one table correspond to values in a primary key column of another related table. It guarantees that the relationships between tables are maintained and prevents the insertion of invalid or inconsistent data.
    a. Foreign Key Constraint: This is a type of referential integrity constraint that defines a relationship between two tables. It ensures that values in a foreign key column of a table refer to valid primary key values in the related table.
    b. Primary Key Constraint: The primary key constraint defines a column or a set of columns that uniquely identify each record in a table. It ensures that the primary key values are unique and not null.

Example

```
CREATE TABLE Employees (
  employee_id INT PRIMARY KEY,
  employee_name VARCHAR(50) NOT NULL,
  employee_email VARCHAR(100) NOT NULL UNIQUE,
  employee_age INT CHECK (employee_age > 0),
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES
Departments(department_id)
);
```

Authorization

Authorization refers to the process of granting or denying access rights to users or roles in a database system. It is an essential component of database security and helps ensure that only authorized individuals can access and manipulate data.

Authorizations on data include:

- Authorization to read data.
- Authorization to insert new data.
- Authorization to update data.
- Authorization to delete data.

Each of these types of authorizations is called a privilege. We may authorize the user all, none, or a combination of these types of privileges on specified parts of a database, such as a relation or a view. When a user submits a query or an update, the SQL implementation first checks if the query or update is authorized, based on the authorizations that the user has been granted. If the query or update is not authorized, it is rejected.

In addition to authorizations on data, users may also be granted authorizations on the database schema, allowing them, for example, to create, modify, or drop relations. A user who has some form of authorization may be allowed to pass on (grant) this authorization to other users, or to withdraw (revoke) an authorization that was granted earlier. Authorization is typically implemented using access control mechanisms, such as access control lists (ACLs) or role-based access control (RBAC).

Privileges are granted to users or roles to perform specific actions or operations on database objects. The GRANT and REVOKE statements are used to grant or revoke privileges, respectively. These statements are part of the SQL (Structured Query Language) syntax and are executed by database administrators or users with sufficient privileges.

The **GRANT statement** is used to grant privileges to users or roles. The basic syntax of the GRANT statement is as follows:

```
grant <privilege list>
on <relation name or view name>
to <user/role list>;
```

The following grant statement grants database users Amit and Satoshi select authorization on the department relation.

```
grant select on department to Amit, Satoshi;
```

The **REVOKE statement** is used to revoke privileges that have been previously granted. The basic syntax of the REVOKE statement is as follows:

```
revoke <privilege list>
    on <relation name or view name>
from <user/role list>;
```

Thus, to revoke the privileges that we granted previously, we write

```
revoke select on department from Amit, Satoshi;
```

In addition to granting and revoking privileges directly to users, database management systems (DBMS) also support authorization on roles and views.

- Authorization on Roles:

Roles are named groups of privileges that can be assigned to users. By assigning privileges to roles and then granting those roles to users, the management of privileges becomes more efficient. This approach simplifies user management and allows for easier maintenance of privileges.

Roles can be created in SQL as follows:

```
create role instructor;
```

Roles can then be granted privileges just as the users can, as illustrated in this statement:

```
grant select on takes
to instructor;
```

Roles can be granted to users, as well as to other roles, as these statements show:

```
grant dean to Amit;
create role dean;
grant instructor to dean;
grant dean to Satoshi;
```

Thus the privileges of a user or a role consist of:

- All privileges directly granted to the user/role.
- All privileges granted to roles that have been granted to the user/role.

Authorization on Views:

Views are virtual tables derived from the underlying tables in a database. They provide a way to present a subset of data or a customized view of the data to users. You can also control access to views by granting privileges on them.

1. Create view
2. Give grant privilege to that view
3. Or revoke that privilege from view

Transfer of privileges

The transfer of privileges refers to the ability to grant or transfer privileges from one user to another. This can be useful in situations where a user needs to delegate certain permissions to another user temporarily or permanently.

By default, a user/role that is granted a privilege is not authorized to grant that privilege to another user/role. If we wish to grant a privilege and to allow the recipient to pass the privilege on to other users, we append the with grant option clause to the appropriate grant command.

For example, if we wish to allow Amit the select privilege on department and allow Amit to grant this privilege to others, we write:

```
grant select on department to Amit with grant option;
```

The creator of an object (relation/view/role) holds all privileges on the object, including the privilege to grant privileges to others.

Revoking of Privileges

Revoking privileges is the process of removing previously granted permissions or privileges from a user or role in a database. This is typically done when the privileges are no longer needed or when there is a change in the access requirements. In SQL, you can revoke privileges using the REVOKE statement.

It's important to note that the ability to revoke privileges may be restricted to certain privileged users or roles. Additionally, some database systems may have additional options or clauses to control the revocation process, such as specifying the option to CASCADE the revocation to dependent objects.

- `revoke select on department from Amit, Satoshi restrict;`
- `revoke grant option for select on department from Amit;`
- `REVOKE SELECT ON employees FROM user1;`

Functions and Procedures

Functions:

- Functions are database objects that accept input parameters, perform a specific computation or calculation, and return a single value.
- They are typically used within SQL statements to perform operations on data and return the result.
- Functions can be used in SELECT statements, WHERE clauses, and other parts of SQL queries.
- Examples of functions include mathematical functions (e.g., SUM, AVG, MAX, MIN), string functions (e.g., CONCAT, SUBSTRING, LENGTH), and date functions (e.g., DATE, MONTH, YEAR).

Procedures:

- Procedures, also known as stored procedures, are similar to functions but can perform multiple operations or tasks.
- They can accept input parameters, execute multiple SQL statements or blocks of code, and may or may not return a value.
- Procedures are often used to encapsulate a series of database operations into a single unit, which can be executed as a single command.
- Unlike functions, procedures do not return a value directly but can use output parameters to pass data back to the calling program or script.
- Procedures can be executed using a CALL statement or from other parts of the database system.
- They are commonly used for complex data manipulations, transaction management, and implementing business logic.

The main difference between functions and procedures is that functions return a value, while procedures do not necessarily return a value. Additionally, functions can be used within SQL queries, whereas procedures are executed as standalone commands.

1. **Declaring and Invoking SQL Functions and Procedures**

Declaring a function:

```
CREATE FUNCTION max_value (table_name in VARCHAR(255),
column_name in VARCHAR(255))

RETURNS INT

BEGIN

  DECLARE max_value INT;

  SELECT MAX(column_name) INTO max_value

  FROM table_name;

  RETURN max_value;

END;
```

Invoking a Function:

```
SELECT max_value ('customers', 'age');
```

Declaring a Procedure:

```
CREATE PROCEDURE add_customer (name in VARCHAR(255), age in INT)

BEGIN

  INSERT INTO customers (name, age)

  VALUES (name, age);

END;
```

Invoking a Prcedure:

```
CALL add_customer ('John Doe', 30);
```

Parameters in SQL functions and procedures are variables used to pass values into the function or procedure. There are three types: **IN parameters** for input values, **OUT parameters** for returning values, and **INOUT parameters** for both input and output. They make functions and procedures more flexible and reusable. IN parameters are read-only, while OUT and INOUT parameters can be modified within the function or procedure.

2. Language Constructs for Procedures and Functions

SQL supports constructs that give it almost all the power of a general-purpose programming language. The part of the SQL standard that deals with these constructs is called the **Persistent Storage Module (PSM).** Variables are declared using a declare statement and can have any valid SQL data type. Assignments are performed using a set statement. A compound statement is of the form **begin . . . end**, and it may contain multiple SQL statements between the begin and the end.

Some of the commonly used constructs include:

- IF: This statement is used to control the flow of execution.
- ELSE: This statement is used to provide an alternative execution path for the IF statement.
- LOOP: This statement is used to create a loop.
- WHILE: This statement is used to create a loop that will continue to execute as long as a condition is met.
- FOR: This statement is used to create a loop that will execute a certain number of times.
- DO: This statement is used to execute a block of code repeatedly.
- BREAK: This statement is used to exit a loop.
- RETURN: This statement is used to return a value from a function.
- EXCEPTION: This statement is used to handle exceptions.

```
DECLARE
    customer_name VARCHAR(255);
BEGIN
    SELECT name INTO customer_name
    FROM customers
    WHERE id = 1;

    IF customer_name IS NOT NULL THEN
        -- Do something with the customer name
    END IF;
END;
```

Triggers

A trigger is a statement that the system executes automatically as a side effect of a modification to the database. To design a trigger mechanism, we must meet two requirements:

- Specify when a trigger is to be executed. This is broken up into an event that causes the trigger to be checked and a condition that must be satisfied for trigger execution to proceed.
- Specify the actions to be taken when the trigger executes.

Once we enter a trigger into the database, the database system takes on the responsibility of executing it whenever the specified event occurs and the corresponding condition is satisfied.

Triggers in SQL are database objects that are automatically executed in response to specified events, such as INSERT, UPDATE, or DELETE operations on a table. Triggers are used to enforce data integrity, implement business logic, or perform additional actions based on the occurrence of certain events.

Triggers can be used to perform a variety of tasks, such as:

- Validating data before it is inserted or updated
- Logging changes to data
- Enforcing business rules
- Generating reports

```
-- Create the trigger

CREATE TRIGGER insert_audit

AFTER INSERT ON employees

FOR EACH ROW

BEGIN

  INSERT INTO employee_audit (employee_id, action_date, action)

  VALUES (NEW.id, CURRENT_TIMESTAMP, 'INSERT');

END;
```

Indexes

Indexes in SQL are database structures that improve the performance of queries by providing quick access to data. They work similar to indexes in a book, allowing you to locate specific information faster.

An index is created on one or more columns of a table. When a query is executed that involves those columns, the database engine can use the index to quickly find the relevant rows, rather than scanning the entire table.

To create an index, you typically use the CREATE INDEX statement, specifying the table name, index name, and the column(s) on which the index should be created. Here's an example:

```
CREATE INDEX idx_lastname ON employees (last_name);
```

In this example, an index named "idx_lastname" is created on the "last_name" column of the "employees" table.

4.0 Structure Query Language (SQL)

SQL was developed in 1970's in an IBM laboratory "San Jose Research Laboratory" (now the Amaden Research center). SQL is derived from the SEQUEL one of the database language popular during 1970's. SQL established itself as the standard relational database language. Two standard organization (ANSI) and International standards organization (ISO) currently promote SQL standards to industry.
In 1986 ANSI & ISO published an SQL standard called SQL-86. In 1987, IBM published its own corporate SQL standard, the system application Architecture Database Interface (SAA-SQL). In 1989, ANSI published extended standard for SQL called, SQL-89. The next version was SQL-92, and the recent version is SQL: 1999.

4.1 Basic Term and Terminology

Query: is a statement requesting the retrieval of information.
Query language: language through which user request information from database. These languages are generally higher level language than programming language.

The two types of query language are:

(i) Procedural language

- User instructs the system to perform sequence of operation on the database to compete the desired result. Example : relational algebra

ii) Non- procedural language

- User describes the desired information without giving a specific procedure for obtaining that desired information.
- Examples: tuple relational calculus and domain relational calculus.

4.2 Database Languages

Two types of database language
1. Data Definition Language (DDL)
2. Data Manipulation Language (DML)

Data Definition Language

- Specifies the database schema.
- For e.g.: The following statement in SQL defines relation named `student'.

```
CREATE TABLE student
(
  student_id  VARCHAR2(3),
  address      VARCHAR(30)
);
```

The execution of this DDL statement creates the `student' table. It also updates a special set of tables called *data dictionary* or *data directory*. A data dictionary contains metadata, that is data about data. The schema of table is an example of metadata. A database system consults data dictionary or data directory.

Through the set of special type of DDL, called data storage definition language, we may specify the storage structure (like size of database, size of table etc) and access methods.

The DDL allow to enforce constrains in the database. For example: student_id should begin with `S', address could not be null etc.

```
CREATE TABLE STUDENT
(
  student_id   VARCHAR2 (3),
  address      VARCHAR2 NOT NULL,
  CONSTRAINT ch_student_id CHECK (student_id LIKE `S%')
 );
```

The database systems check these constraints every time the database is updated.

Data Manipulation Language:

- A data manipulation language is a language that enables users to access or manipulate the data in database. The data manipulation means :
  - Retrieval of information stored in database.
  - The insertion of new information into database.
  - Deletion of information from database.
  - Modification of data in database.

Two types of data manipulation languages are:

- Procedural DML: User need to specify what data are needed to retrieve (modify) and how to retrieve those data.
- Non Procedural (Declarative DML) : User requires to specify what data are needed to retrieve without specifying how to get (retrieve) those data.
  - Non procedural DML are easier to understand and use than procedural DML, since user does not have to specify now to get data from database.
  - The DML component of SQL is non procedural language.

Example: Consider a simple relational database.

customer_id	customer_name	customer_address
C001	Smith	Kathmandu
C002	John	Bhaktapur
C003	Semi	Lalitpur
C004	Ivan	Kathmandu

The customer table

account_no	balance
A101	900
A102	300
A103	200
A104	700

The account table

customer_id	account_no
C001	A001
C002	A002
C003	A004
C004	A004

The depositor table

Some queries and their equivalent SQL statement

Query: Find the name of customer whose customer_id C001.

```
SELECT customer. customer_name FROM customer
        WHERE customer.customer_id = ` C001';
                OR
SELECT customer_name FROM customer
      WHERE customer_id = ` C001';
```

Note: We don't need to specify the table name while referencing column_name if we are taking column from only one table.

Query: Find the name and balance of the customer.

```
SELECT customer.customer_name,account.balance
        FROM customer,account.balance
                WHERE customer.customer_id= depositor.customer_id
                    AND depositor.acount_no = account.account_no;
```

Problem : Insert record to customer table.

customer_id : C005
customer_name : MICHAEL
address : KATHMANDU

```
INSERT INTO customer (customer_id, customer_name, address)
        VALUES (`C005' , `MICHAEL' , `KATHMANDU') ;
                                OR
INSERT INTO Customer values (`C005' , `MICHAEL' , `KATHMANDU');
```

Note: Column name need not to specify if we are going to insert values for all columns of table.

Query: Delete record from depositor whose customer_id is `C004'.
:

```
DELETE FROM depositor  WHERE Customer_id = `C004';
```

What happen if we execute DELETE statement as below?

```
DELETE FROM depositor;
```

- Deletes all records from the table `depositor'

Problem: If you attempt to delete all records of customer from customer table, what happen ?

- You cannot delete all records, only when "account" and "deposit" tables are empty or only when these table contains records that are not related to the customer.

Query: Increase the balance by 5% in account table whose account no is `A101' or current balance is only 200.

```
UPDATE account
        SET balance = balance + (balance + 0.05)
WHERE account_no = `A1001' OR balance = 200;
```

Note:

Sometimes database languages are also categorized with Data Control Language.

That is

1. Data Definition Language (DDL)
2. Data Control Language (DCL)
3. Data Manipulation (DML)

Data Control language is a language that controls the behavior of database.
In SQL, COMMIT, ROLLBACK, commands go under Data Control Language.

- COMMIT: Saves the changes made to database.
- ROLLBACK: Undo changes to database from the current state database to last commit state.

Question: Why we need query language, even we have formal languages?
(formal languages, relational algebra, relational calculus)

- The formal languages provides concise notation for representing query. But commercial database system requires more user friendly query language. This is a main reason for why we need query languages, even we have formal languages.
- The formal languages form the basis for data manipulation language of DBMS only but DBMS/commercial DBMS also supports data definition capabilities as well as data manipulation capabilities.
- SQL is a most popular and powerful query language. It can do much more than just query database. It ca define structure of data, modify data in database and allow to specify security constraints.
- SQL is a truly non procedural language. It has all features of relational algebra, relational calculus as well as its own powerful features.

4.3 Different parts of SQL Language

1. Data Definition Language (DDL):

SQL DDL provides commands for defining relation schemas, deleting schema, deleting relations and modifying relational schemas.

Example:
CREATE, ALTER, DROP

```
CREATE TABLE dept
(
 dept no   NUMBER(2) PRIMARY KEY,
 dname    VARCHAR2(20) NOT NULL
);

CREATE TABLE emp
(
  empno NUMBER(5) PRIMARY KEY,
  deptno NUMBER(3),
  ename VARCHAR2(10) NOT NULL,
  sal NUMBER(5) NOT NULL,
  CONSTRAINT fk_emp_dept FOREIGN KEY(deptno) REFERENCES dept
```

);

```
ALTER TABLE dept ADD (loc VARCHAR2(10));
ALTER TABLE emp MODIFY (empno NUMBER(10));
ALTER TABLE emp ADD UNIQUE (ename);
DROP TABLE emp;
DROP constraint fk_emp_dept;
```

2. Data Manipulation Language (DML):

The SQL DML includes query language based on relational algebra and relational calculus. It includes commands for insert tuples, delete tuples and modify tuples in database.

Example: INSERT, DELETE, UPDATE, SELECT etc. statements.

3. View Definition:

The SQL DDL includes for defining views e.g.

Syntax:

```
CREATE VIEW <view name> AS
  (<query expression>);
```

4. Transaction Control: (Data Control Language):

SQL includes commands for specifying integrity constraints that the data stored in database must satisfy.

5. Embedded SQL and dynamic SQL:

Embedded SQL & dynamic SQL dynamic SQL is that SQL with general purpose programming language; such as C, C++, JAVA, COBAL, PASCAL, FORTRAN.

6. Integrity:

SQL DDL includes commands for specifying integrity constraints that the data stored in database must satisfy.

7. Authorization:

SQL DDL commands used for specifying access right to relation relations and views.

4.4 General overview of SQL:

Though SQL user / programmer / DBA can perform the following task:

- Create database.
- Modify a database structure.
- Add user permissions to database or tables.
- Changes system security.
- Query a database for a retrieval of information.
- Updates the contents of information.

Note: Commands in SQL are not necessarily a question, request to the database. It could be a command to do one of the following.

- Build or delete a table.
- Insert, Modify or delete rows or fields.
- Search several tables for specific information and returns the result in specific order.
- Modify security information.

Note: Commands in SQL are not case-sensitive. But generally conversation is write a keywords as a capital and other should be in small letter.

Data Manipulation Language in SQL:

SQL provides the following basic data manipulation statements: SELECT, UPDATE, DELETE and INSERT.
The select statements:

- The SELECT statement is most commonly used SQL statement. It is only a data retrieval statement in SQL.
- The basic syntax for select statement is

```
SELECT  [DISTINCT / ALL ] <attributes> 1
  FROM <relations> 2
[WHERE <predicate>] 3
```

1. <attribute> -> columns name
2. <relations> -> tables name
3. <predicated> -> conditions

- SELECT, FROM are necessary clause.
- WHERE is optional clause.
- DISTINCT / ALL are optional clause.
- SELECT clause used to list the attributes that required in the result in query.
- FROM clause list the relation/s from where specified attributes are to be selected.
- WHERE clause are used to specify the condition/s while we require retrieving particular data. One or more condition can be specified using where clause by using SQL logical connectives can be any comparison operators <, <=, >, >=, = and < >. SQL also includes BETWEEN comparisons.
- DISTINCT key word is used to eliminate duplicate value.
- ALL key word is used to explicitly allow duplicates.

Example: Assumed simple relational database is as follows.

customer_id	customer_name	customer_address
C001	Smith	Kathmandu
C002	John	Bhaktapur
C003	Semi	Lalitpur
C004	Ivan	Kathmandu

The customer table

account_no	balance
A101	900
A102	300
A103	200
A104	700

The account table

customer_id	account_no
C001	A001
C002	A002
C003	A004
C004	A004

The depositor table

a. Find all customer names.

```
SELECT customer_name FROM customer;
```

b. Find the different customer address (location).

```
SELECT DISTINCT customer_address FROM customer;
```

c. Find all address of customer.

```
SELECT ALL customer_address FROM customer;
```

d. Find customer_id and its corresponding customer name.

```
SELECT customer_id, customer_name FROM customer;
```

e. Find customer detail.

```
SELECT *FROM customer    (* indicates all attributes)
```

f. List name and address of customer who stay in "KATHMANDU".

```
SELECT customer_name, customer_address, FROM customer
        WHERE customer_address = "KATHMANDU";
```

g. List all customer whose name should be "smith" and address should be "Kathmandu".

```
SELECT customer_name FROM customer
        WHERE customer_name = 'smith'
          OR customer_ address  =  'Kathmandu'
```

h. What would be the output if statement like

```
SELECT customer_name FROM customer
        WHERE customer_name FROM customer
          OR customer_ address = 'Kathmandu'
```

i. List account no, balance whose balance is between 200 to 700.

```
    SELECT account_no, balance FROM account
        WHERE balance BETWEEN 200 AND 700;
```

j. What happen if we execute the statement?

```
    SELECT account_no, balance FROM account
        WHERE balance NOT BETWEEN 200 AND 700;
```

k. Write SQL statement for (i) using only AND logical connectives and comparison operatives.

```
  SELECT  account_no, balance FROM account
        WHERE balance <= 700 AND balance >= 200;
```

Note: We can retrieve the information from multiple tables; there should be a common attribute between two tables. i.e., table should be related and we require to join condition.

l. List the customer_id, account_id and balance whose balance is more than 300.

```
SELECT depositor. customer_id, depositor. account_no, account. balance
    FROM depositor, account
        WHERE depositor.account_no = account.account_no
            AND account.balance > 300;
```

m. List all customers and corresponding balance.

```
SELECT c.customer_name, a.balance
            FROM customer   c, account    a, depositor    d
        WHERE d.account_no =a.account_no
            AND d.customer_id =c.customer_id;
```

Renaming attribute and relations:

- In previous example relations customer, account, depositor are renamed respectively c, a and d.

- We can also rename attributes, it is required when we are taking attribute from multiple column or we need arithmetic operations in the statement or when we need to give appropriate name for (column name) attribute name.
- To rename attribute SQL provides as clause or we can simply rename attribute or relation without as clause.

Examples:

a. SELECT account_no as account number FROM account;

OR

SELECT account_no "Account number" FROM account;

b. SELECT account_no, balance, balance+ (balance*0.05) as Account Number, Balance,
Increase salary FROM account;

OR

SELECT account_no "Account number", balance "Balance:,
Balance+(balance*0.05) "Increased salary" FROM account;

c. SELECT c. customer_name, a. balance
FROM customer c, account a, depositor d
WHERE d. account_no = a. account_no
AND d.customer_id = c.customer_id;

OR

SELECT c. customer_name, a. balance
FROM customer as c, account as a, depositor as d
WHERE d.account_no = a.account_no
AND d.customer_id = c.customer_id;

String operations:

String pattern matching operation on SQL can be performed by `like' operator and we can describe the patterns by two spherical character.

1. Percent (%) : matches any substrings.
2. underscore (_): matches any characters.

Example:

`I%' matches any string beginning with I.
IVAN -> valid / match.
INDIA -> valid / match
NEPALI -> invalid / does not match.

` % VAN% match any string containing `VAN' as substring.
IVAN, Mr IVAN, DEVAN, DEVANGAR are all valid.
`---' matches any strings of exactly three character.
`---%' matches any string of atleast three character.

Moreover,

Like `ab\%cd%' matches all string beginning with "ab%cd".
Like `ab\\cd%' matches all string beginning with "ab\cd".

Problems:

List those customers whose name begin with character `S'

```
SELECT customer_name FROM customer.
        WHERE customer_name LIKE `S%';
```

Note:

customer_name like `S%H'

- List all customer name whose name begin with `S' and end with `H'

customer_name like`----'%N'

- List all customer name whose name must contain at least five character and end with character `N'

Ascending and descending records in SQL:

- ORDER BY clause used for ascending or descending records(or list items).
- To specify sort order we may specify desc for descending order or asc ascending orders. By default order by clause list item in ascending order.
- Moreover, ordering can be performed on multiple attributes.

Example: 1

```
SELECT distinct customer_name FROM customer ORDER BY customer_name;
```

- Lists name of customer in alphabetic order by customer name.

Example: 2

```
SELECT DISTINCT customer_name FROM customer ORDER BY customer_name
DESC;
```

- Lists name of customer in descending alphabetic order.

Note: select from customer order by 2;

Here 2 indicates second column in table "customer". This SQL statement is equivalent to first example's SQL statement.

Example: 3

Suppose want list account information in descending order by balance but if say some balance are same and in such case if we want to order account information by order_no in ascending order then we have order record by performing ordering on multiple attributes. The SQL statement likes,

```
SELECT *FROM account
    ORDER BY balance DESC, account ASC;
```

Set operation

- basic set operation are union(u), intersection(n) and difference(_). These operation also can be performed by using union, intersection and minus (except) clause respectively.

The union operation

- the union operation can be perform by using union clause.

Example: consider two reactions

client_id	name
C001	Ammit
C002	Ajay
C003	Rohit
C004	Ammit

The table client

supplier_id	name	city
S001	ASHOK	Kathmandu
S002	MANISH	Bhaktapur
S003	MANOJ	Kathmandu
S004	MANISH	Bhaktapur

The table supplier

List the id and name of the client and supplier who stay in city 'Kathmandu'.

```
Select supplier_id  "ID", name "Name" from supplier
        where city = 'Kathmandu'
UNION
SELECT client_id "ID", name "Name" from client
       where city = 'Kathmandu'
```

- Proceed as follow:
  Output from 1st SQL statement

ID	Name
S001	Ashok
S002	Manoj

Output from 2nd SQL statement

ID	Name
C001	Ammit
C002	Ammit

Hence, the resulting output is

ID	Name
C001	Ammit
C002	Ammit
C003	Ashok
C004	Manoj

Note: if we retrieve only one column say name without duplicate name then corresponding statement like

```
Select name from supplier where city = 'Kathmandu'
UNION
Select name from client where city = 'Kathmandu';
```

- this is unlike select clause, union operation automatically eliminates duplicates. If we want to retain all duplicates, we must replace union all.

Example:
select name from supplier where city = 'Kathmandu'.

The output would be

Name
Ammit
Ammit
Ashok
Manoj

The intersection operation

- the intersection operation can be performed by using INTERSECT clause
- consider relation as follow:

SALESMAN

salesman_id	name	city
S001	Manish	Kathmandu
S002	Manoj	Lalitpur
S003	Ammit	Bhaktapur
S004	Rabin	Kathmandu

The salesman table

order_no	Order_date	salesman_id
0001	10-JAN-98	S001
0002	12-FEB-98	S002
0003	13-FEB-98	S001
0004	18-MAR-98	S001
0005	19-MAR-98	S002

The sales_order table

Retrieve salesman name who stay in Kathmandu and who must sales at least order.

```
SELECT salesman_id, name
    from salesman
        where city = 'Kathmandu'

INTERSECT
 SELECT salesman. salesman_id
    from salesman, sales_order
        Where salesman_id = sales_order. salesman_id;
```

salesman_id	name
S001	Manish
S004	Rabin

=

salesman_id	name
S001	Manish
S002	Manoj
S001	Manish
S002	Manoj

The resulting output is

salesman_id	name
S001	Manish

- The INTERSECT operation also automatically eliminates duplicates. So, here only one record is delayed in output. If we want to retain all duplicates we must replace INTERSECT by INTERSECT ALL.

```
SELECT salesman_id, name from SELECT salesman
    where city = 'Kathmandu'
INTERSECT ALL
SELECT salesman. salesman_id, salesman name
        from salesman, sales_order
WHERE salesman. salesman_id = sales_order salesman_id;
```

The difference operation

The difference operation can be perform in SQL by using except or minus clause.

Example: in previous example, find the salesman_id, name who stay in Kathmandu but they do not sales any order.

```
SELECT salesman_id, name from salesman
   where city = 'Kathmandu'
EXCEPT
SELECT salesman. salesman_id, salesman name
   from salesman, sales_order
        Where salesman. salesman_id = sales_order. salesman_id;
                  OR
SELECT salesman_id, name from salesman
     where city = 'Kathmandu'
MINUS
SELECT salesman. salesman_id, salesman name
       FROM  salesman, sales_order
            Where salesman salesman_id = sales_order salesman_id;
```

The output is

salesman_id	name
S004	Rabin

NOTE: Except operation also automatically eliminates duplicates so it want to return all duplicates, we must write EXCEPT ALL instead of EXCEPT.

Problem: consider a relation schema as follow

```
branch(#branch_name, branch_city, assets)
account(#account_number, branch_name, balance)
customer(#customer_name,  customer_street, customer_city)
depositor(customer_name, account_number)
loan(#loan_number, branch_name, amount)
brrower(customer_name, loan_number)
```

1. Find all customer who have a loan, account or both at the bank .

```
SELECT customer nameFrom depositor
 UNION
SELECT customer name  From borrower;
```

2. Find all customers who have both loan and account at the bank.

```
SELECT DISTINCT customer name from depositor
     INTERSECT
SELECT DISTINCT customer name from borrower;
```

3. Find all customers who have account but no loan at the bank.

```
SELECT DISTINCT customer name from depositor
     EXCEPT
SELECT customer name from borrower;
```

Aggregate Function

Aggregate functions are those functions that take a set of values as input and return a single value. SQL consist many built in aggregate function. Some are:

1. AVERGE: AVG
2. MAXIMUM: MAX
3. MINIMUM: MIN
4. TOTAL: SUM
5. COUNT: COUNT

The input to AVG and SUM must be a set of numbers and other aggregate function can be operate by non numeric data types, it may be strings, not necessary numbers.

AVG:

Syntax: AVG (<DISTINCT\ALL>; n)

- returns average of n, ignoring null values.

Example: find the average balance in Kathmandu branch.

```
SELECT AVG (balance)  From account
   WHERE branch_name = 'KATHMANDU';
```

There would be a situation that we may have to use aggregate function not only a single set of tuples we may have to use with group of set of tuples, we can specify this by using GROUP BY clause in SQL. That is, group by clause specifies group rows based on distinct values that exist in specified column when we use GROUP BY clause we can not use WHERE clause to specify condition, we must have to use HAVING clause to specify the condition. That is, GROUP BY and HAVING clause. But GROUP BY or HAVING clause act on record sets rather than individual records.

Example: find the average account balance at each branch

```
SELECT branch_name , avg (balance)
    FROM  account
GROUP BY branch_name;
```

MIN:

Syntax: MIN (<DISTINCT\ALL>; n)

- returns minimum value of n.

Example: find the minimum balance of each branch.

```
SELECT branch_name, min (balance) "Minimum Balance"
   FROM account
GROUP BY branch_name;
```

MAX:

Syntax: MAX (<DISTINCT ALL>; n)

- returns maximum value of n

Example: find the maximum balance in each branch.

```
SELECT branch_name, max (balance) "Maximum Balance"
      FROM account
```

```
GROUP BY branch_name;
```

COUNT:

Syntax: COUNT (<DISTINCT ALL>; n)

- returns numbers of rows where n is not null.

NOTE: COUNT (*)

- returns numbers of rows in the table, including duplicates and those with nulls.

Example: find the numbers of depositor for each branch.

NOTE: each depositor may have numbers of account so we must count depositor only once thus we write query as below:

```
SELECT branch_name, count (DISTINCT Customer_name)
    From depositor, account
        WHERE depositor account number = account. Account_number
            GROUP BY branch_name;
```

NOTE: if we need to specify the condition (predicates) after GROUP BY clause, we need having clause.

PROBLEM: find only those branches where the average account balance is more than 1200.

```
SELECT branch_name, AVG (balance) FROM account
 GROUP BY branch_name having AVG (balance) > 1200;
```

PROBLEM: find the no. of customer in customer table.

```
SELECT COUNT (*) FROM Customer;
```

NOTE: If a WHERE clause and HAVING clause both appears in the same query, SQL executes predicate in the WHERE clause first and if it satisfied the only it executes predicate of GROUP BY clause.

PROBLEM: Find the average balance for each customer who lives in KATHMANDU and has at least three accounts.

```
SELECT depositor customer_name, AVG (balance)
    FROM depositor, account , customer
        WHERE depositor account_number = account account_number
            Depositor customer_name = customer customer_name
            Customer_city = 'KATHMANDU'
GROUP BY depositor, customer_name
  HAVING COUNT (DISTINCT depositor account_number) > = 3;
```

SUM

Syntax: SUM ([DICTINCT/ ALL] n)

- return sum of value of n

Example: find total loan amount for each branch

```
SELECT branch_name, SUM (amount)
    FROM loan
GROUP BY branch_name;
```

NULL VALUES

- In SQL, NULL values all to indicate absence of information for the value of attribute.
- SQL provides special key word NULL in a predicate to test for NULL values. Not NULL in predicate use to test absence of NULL values.

Example: find the balance of each branch whose balance is not empty.

```
SELECT branch_name, balance
        FROM account
WHERE balance is NOT NULL;
```

Example: List the account number, branch name and balance whose balance is empty.

```
SELECT *FROM account
    WHERE balance is NULL;
```

- The feature of SQL that handles NULL values has important application but some time it gives unpredictable result. For example, an arithmetic expression involving (+. _ , * or /), if any of the input values is NULL value (except IS NULL Q IS NOT NULL).
- If NULL values exist in the processing of aggregate operation, it makes process complicated.
  Example: SELECT AVG (amount) FROM loan;
- This query calculates the average loan amount that is not empty so if there exist empty amount then calculated average amount is not valid. Except COUNT(*) function all aggregate function ignores NULL values in input.
- The COUNT ( ) function return if count value is empty and all other aggregate function returns NULL if it found empty value.

Example: consider a table 'emp' as below:

Empno	Sal	Comm
10	100	
20	200	50
30	300	20

Suppose the SQL statement are as below

a. SELECT COUNT (*) FROM emp;
   returns count is equal to 3
   That is count(*)

   3

b. SELECT COUNT (comm.) FROM emp WHERE empno = 10;
   returns count(comm.)

   0

c. SELECT SUM(COMM) FROM emp WHERE empno = 10;
   return sum(comm.)

   (nothing)

d. SELECT SAL+COMM FROM emp WHERE empno = 10;

```
return          sal+comm.
                ----------
                  (nothing)
```

- here the result is unpredictable. In this case result should be 100. To handle such unpredictable situation SQL provides NVL ( ) function
  Example: SELECT sal+nvl(comm,0)/100

- nvl function returns 0 when comm is found empty. If user do not specify the value for any column (attribute) SQL place null values in these columns. The null value is different from zero. That is null value is not equivalent to value zero.
- A NULL value will evaluate to NULL in any expression.
  Example: NULL multiply by 10 is NULL.
- If the column has a NULL value, SQL ignores the unique Foreign key, check constraints that are attached to the column.
- If any field define as NOT NULL, it does not allow to ignore this field, user must insert value, that is NOT NULL is itself a constraint while it specify in table.

Nested Subqueries

- SQL provides sub_query facility. A sub_query is a SQL statement that appears inside another SQL statement. It is also called nested sub_queries or simply nested query.
- Sub_query use to perform tests for set membership, make set comparisons and determine set cardinality.
- Sub_query can be used with SELECT, INSERT, UPDATE and DELETE statement.

Example: find all branch name where depositor account number is ‘A005’ .

```
SELECT branch name FROM account
      WHERE account account_number = (SELECT account_number FROM
depositor
                                          WHERE account number
= ‘A005’);
```

- When sub_query return more than one values the we required to test whether value written by first query is match/exist or not within values return by sub_query.
- IN and NOT IN connectives are useful to test in such condition. That is IN connectives use to test for set membership and NOT IN connectives use to test for absence of set membership.

Example: find those customers who are borrowers from the bank and who are also account holder.

```
SELECT DICTINCT customer_name FROM borrower
  WHERE customer_name IN (SELECT customer_name FROM
depositor);
```

Example: find all customer who do have loan at the bank but do not have an amount at the bank.

```
SELECT DISTINCT customer_name FROM borrower
  WHERE customer_name NOT IN (SELECT customer_name FROM
depositor);
```

Example: list the name of customers who have a loan at the bank and whose name neither SMITH nor JONE

```
SELECT DISTINCT Customer_name FROM borrower
  WHERE customer_name NOT IN ('SMITH', 'JONE');
```

Example: find all customers who have both an account and loan at Kathmandu branch.

```
SELECT DISTINCT Customer_name FROM borrower, loan
  WHERE borrower loan number = loan loan_number and
branch_name = 'KATHMANDU'
      AND (branch_name, customer_name) IN (SELECT
branch_name, customer_name
                FROM depositor account
                  WHERE depositor account_number = account
account_name);
```

SET COMPARISION

Nested sub_query have an ability to compare set.

Example: Find the names of all branches that have assets greater than those of at least one branch located in 'Kathmandu'.

The simple SQL statement is

```
SELECT DISTINCT B1 branch_name FROM branch B1. branch B2
  WHERE B1 assets>B2 assets
    AND B2.branch_city = 'Kathmandu'.
```

The same query can be written by using subquery as

```
SELECT branch_name FROM branch
  WHERE assets > some (select assets FROM branch
                       WHERE branch_city = 'Kathmandu');
```

- here, >some comparison in the where clause of the outer value return by sub_query.
- SQL also allow <some, >=some, =some and < > some comparison
- Not that = some is identical to IN. but < > some is not same as NOT IN.

Example: Find the names of all branches that have an assets value greater than that of each branch in 'KATHMANDU'.

NOTE: The construct > all corresponds to the phase 'greater than all'

```
SELECT branch_name FROM branch
  WHERE assets > all (SELECT assets FROM branch
                      WHERE branch city = 'KATHMANDU')
```

NOTE: SQL also allow <all, <=all, >=all, =all and < >all comparison.

Example: find the branch that has the highest average balance.

NOTE that we can not use MAX {AVG (balance)}, since aggregate function can not be composed in SQL. So we first need to find all average balances and need to nest

it as subquery of another query that finds those branches for which average balance is greater than or equal to all average balances.

```
SELECT branch_name FROM account
    GROUP BY branch_name
  HAVING AVG (balance) >=all (SELECT AVG (balance) FROM account
                                                    GROUP BY
branch_name);
```

TEST EMPTY RELATIONHIP

SQL has a feature for testing whether a subquery return any value or not. The exists construct returns true if subquery returns values.

Example: find all customers who have both an account and loan at the bank.

```
SELECT customer_name FROM borrower
WHERE exists (SELECT *FROM depositor
                        WHERE depositor customer_name = borrower
customer_name);
```

We can test non existence of values (tuples) in sub_query by using not exists construct.

Example: find all customers who have an account at all branches located in 'KATHMANDU'.

```
SELECT DISTINCT d1.customer_name
  FROM depositor as d1
WHERE not exists {(SELECT branch_name FROM branch
                        WHERE branch_city = 'KATHMANDU')
Except
 (SELECT d2. branch_name FROM depositor as d2, account as a
            WHERE d2. Account–no. = a. Account_no.
                AND d1. Customer_name = d2.
Customer_name)};
```

Test for the absence of Duplicate Tuples

SQL has a feature for testing whether the subquery has any duplicate Tuples in its results.
The UNIQUE construct true if a subquery contains no duplicate Tuples.

Example: find all customers who have at most one account at the KATHMANDU branch.

```
SELECT d.customer_name FROM depositor d1
    WHERE UNIQUE (SELECT d2. customer_name FROM account depositor d2
                WHERE d1. customer_name = d2. customer_name
                  AND d2. account_no. = account. Account_no.
                    AND account. Branch_name = 'KATHMANDU');
```

NOTE: using NOT UNIQUE construct, we can test the existence of duplicate tuples.

Example: find all customers who have at least two account at the KATHMANDU branch.

```
SELECT DISTINCT d1.customer_name FROM depositor as d1
        WHERE UNIQUE (SELECT d2. customer_name FROM account depositor d2
                                WHERE d1. customer_name = d2. customer_name
                                  AND d2. account_no. = account. Account_no.
                                    AND account. Branch_name = 'KATHMANDU');
```

Complex Queries

There are several way of composing query: derived relation and the with clause are ways of composing complex queries.

Derived Relation

- SQL have a feature that it allow sub_query expression to used in the FROM clause.
- If we use such expression then we must give result relation name and we can remove the attributes.

Example: find the average account of those branches where the average account balance is greater than 1200.

```
SELECT branch_name, avg_balance
    FROM {SELECT branch_name, avg (balance) FROM account GROUP BY
branch_name}
AS branch_avg (branch_name, avg_balance)
     WHERE avg_balance > 1200;
```

The with clause

- The with clause introduced in SQL: 1999 and is currently supported by only some database.
- The with clause makes query logic clear.

Example: find all branches where the total account deposit is less than the average deposits at all branches.

```
WITH branch_total (branch_name, value) as
 SELECT branch_name, SUM (balance) FROM account
     GROUP BY branch_name
  WITH branch_total_avg (value) as
      SELECT avg (value) FROM branch_total
   SELECT branch_name FROM branch_total, brnch_total_avg
     WHERE branch_total.value >= branch_total_avg.value;
```

UPDATE STATEMENT

- The UPDATE statement is used to modify one or more records in specified relation. The records to be modify are specified by a predicate in the WHERE clause and new value of the column (s) to be modified is specified by a SET clause.

The syntax is

```
UPDATE <relation>
    SET <attribute with new value>
[WHERE <predicate>];
```

Example: increase the balance of all branches by 5%

```
UPDATE account
```

```
SET balance = balance * 1.05;
            OR
UPDATE account
    SET balance = (balance) + (balance * 0.05);
```

Example: increase balance of those branches whose current balance is less than or equal to 1000 by 5%

```
UPDATE account
    SET balance = balance * 1.05
  WHERE balance <= 1000;
```

Example: decrease the balance by 5% on accounts whose balance is greater than average.

```
UPDATE account
SET balance = balance_ (balance * 0.05)
WHERE balance > {SELECT average (balance) FROM account};
```

NOTE: SQL provides case construct, which can be perform multiple updates with a single UPDATE statements

The syntax is:

```
Case
When predicate 1 then result 1
When predicate 2 then result 2
When predicate n then result n
        Else result
END
```

Example:

```
UPDATE account
    SET balance = case
                  When balance <= 1000then balance *1.05
                  else balance *1.06
                  end;
```

DELETE STATEMENT

The DELETE statement use to delete one or more records from relations. The records to be deleted are specified by the predicate in the WHERE clause.

The Syntax is:

```
DELETE <relation> [WHERE <Predicate>]
```

Note: Delete statement can operate only one relation. It can not delete records of multiple relation.

Example: Delete all records from loan relation.

```
DELETE FROM loan;
```

Example: Delete all records from account relation whose branch is located in Kathmandu.

```
DELETE FROM account
WHERE branch_name = 'KATHMANDU;
```

Example: Delete all loan with loan amount between 1000 and11500;

```
DELETE FROM account
WHERE branch_name IN (SELECT branch_name FROM branch)
                      WHERE branch_city = 'KATHMANDU');
```

Example: Delete all records of account with balance below the average

```
DELETE FROM account
WHERE balance <(SELECT avg (balance) FROM account);
```

INSERT STATEMENT

The INSERT statement used to insert a new records into a specified relation

Syntax:

```
INSERT INTO <relation>
    VALUES (<values list>)
```

Another form:

```
INSERT INTO <relation> (<target columns>)
    VALUES (<values list>)
```

The attributes values that are going to insert must be match order by corresponding attributes and also must be matched data type of value and corresponding attribute data type.

Example: insert one record in account relation.

```
INSERT INTO account
  VALUES ('A_0009', 'LALITPUR', 1500);
                  OR
   INSERT INTO account (account_no., branch_name, balance )
 VALUES ('A_0009', 'LALITPUR', 1500);
```

we can also insert records on the basis of query

Example: INSERT INTO account

```
        SELECT loan_number = 'KATHMANDU';
```

It inserts the records in account relation taking account_number, brancg_name from loan relation whose branch is located in Kathmandu and for all records balance is constant 500.

Example: INSERT INTO depositor

```
SELECT customer_name, loan_number FROM borrower, loan
 WHERE borrower . loan_number = loan . loan_number
   AND branch_name = 'KATHMANDU';
```

Insert Tuple (customer_name, loan_number) into the depositor relation for each customer who has a loan in Kathmandu branch with loan number.

Example: INSERT INTO account

```
    SELECT *FROM account;
```

Insert infinite number of Tuples.

Joined Relations

- one of the most powerful feature of SQL is its capability to gather and manipulate data from several relations.

- If SQL does not provides this feature we must have to store all the data elements in a single relations for each application. We have to store same data in several relations.
- The join statements of SQL enables to design smaller, more specific relations that are easier to maintain than larger relations.
- There are several methods for joining relations. Some methods are not useful for application and some are very useful.

1. Cross Join

Joins two or more tables without relation between them or without condition in the where clause. The results is the Cartesian product of two or more table's attributes

Examples: consider two relations

Table 1:

row	remarks
1.	Table 1
2.	Table 1

Table 2:

row	remarks
1.	Table 2
2.	Table 2

The cross join statement is

```
SELECT *FROM Table 1,Table 2;
```

Output is:

row	remarks	row	remarks
1	Table 1	1	Table 2
1	Table 1	2	Table 2
2	Table 1	1	Table 2
2	Table 1	2	Table 2

- cross join is normally not useful but it illustrates the basic combining property of all join types.

Equi Join (natural inner join)

Equi joins methods joins relations based on the equality. It has very important application in commercial database application.

Example: consider

Relation : dept

#empno	E name	job	sal	comm	Dept. no
E001	SMITH	Manager	7000	500	10
E002	JONE	Engineer	6000	3000	20
E003	MICLE	Engineer	5000	2000	20
E004	JACK	Accountant	3000	500	40

relation: emp

#dept. no	Depo. name	Loc
10	Management	Kathmandu
20	Technical	Kathmandu
30	Marketing	Bhaktapur
40	Account	Lalitpur

Example: list all employee name, department name and salary

```
SELECT e. ename, d. dname FROM emp e, dept d
      WHERE  e. deptno = d. deptno;
```

Output:

ename	Dname
SMITH	MANAGEMENT
JONE	TECHNICAL
MICLE	TECHNICAL
JACK	ACCOUNT

We can further qualify this query by adding more condition on where clause.

```
Example: SELECT e.ename, d.dname
              FROM emp e , dept d
       WHERE e.deptno = d.deptno
            AND e.sal >5000       ORDER By e.ename;
```

Example: consider relations

Loan_number	Depo. name	Loc
L_170	Management	Kathmandu
L_230	Technical	Kathmandu
L_260	Marketing	Bhaktapur

Relation: loan

Customer_name	Loan_nuber
JONE	L_170
SMITH	L_230
MICLE	L_155

relation: borrower

```
SELECT *FROM loan, borrower
  WHERE loan. Loan_number = borrower.loan_number;
```

Non – Equi join (Outer join)

SQL also supports non equi_join. That is, this method joins tables (relations) based on non equality. But equi_join is far more common than non equi_join

```
e.g. Select e.ename, d.dname, d.dname d.deptno
                FROM emp e, dept d
       WHERE e.deptno > d.deptno;
```

Output:

Ename	dname	dept no
JONE	MANAGEMENT	10
MICLE	MANAGEMENT	10
JACK	MANAGEMENT	10
JACK	TECHNICAL	20
JACK	MARKETING	30

Here, this information is not so useful.

How from the output? Lets look

```
Case 1:  equi_join
  SELECT e.ename, d.dname, e.deptno "emp deptno", d.deptno "deptno"
    FROM emp e, dept d
  WHERE e.deptno;
```

OUTPUT:

ename	dname	emp deptno	deptno
SMITH	MANAGER	10	10

JONE	TECHNICAL	20	20
MICLE	TECHNCAL	20	20
JACK	ACCOUNT	40	40

REMARKS: output is selected from the Cartesian product of two relations
Emp (e.ename, e.deptno)
Dept (d.dname, d.deptno)
such that both department no is equal only.

Case 2: (Non equi join)

```
SELECT e.ename, d.dname, e.deptno "emp deptno", d.deptno "dept deptno"
    FROM emp e, dept
        WHERE e. dept> d.dept no;
```

Output:

ename	dname	empdeptno	detno
JOHN	Management	20	10
MICHAEL	Management	20	10
JACK	Management	40	10
JACK	Technical	40	20
JACK	Markeing	40	30

..

Remarks:

Output is seleted from the Cartesian product of two relations
emp (e.ename, e.dept no)
Dept (d.dname, d.deptno)
Such that emp tanle of department no is greater than department table of department no.

Case 3 (Non equi join)

```
SELECT e.ename , d.dname  e.dept no "emp deptno"
    FROM emp e ,  dept d
WHERE e. deptno> 10;
```

Output:

ename	dname	Emp deptno	
JONE	MANAGEMENT	20	>10
MICLE	MANAGEMENT	20	>10
JACK	MANAGEMENT	40	>10
JONE	TECHNICAL	20	>10

REMARKS: output is selested from the Cartesian product of two relation
Emp (e.ename, e.deptno)
Dept (e.ename, e.deptno)
Such that emp table of deptno is always greater than 10.

Three types of outer join

1. Left outer join
2. Right outer join
3. Full outer join

Consider two relations

Loan

Loan_no.	Branch_name	amount
L-170	KATHMANDU	3000
L-230	BHAKTAPUR	4000
L-260	LALITPUR	1700

Borrower

Customer_name	Loan_number
JONE	L-170
SMITH	L-230
MICLE	L-155

a. SELECT loan . loan_number, loan . branch_name, loan . amount, borrower . loan_number loan

Left outer join borrower on loan . loan_number = borrower . loan_number

Loan_no.	Branch_name	amount	Customer_name
L-170	KATHMANDU	3000	JONE
L-230	BHAKTAPUR	4000	SMITH
L-260	LALITPUR	1700	null

Remarks: Tuple from the left hand side relation that do not math any Tuple in the right side relation are padded with null.
SELECT loan. loan_number, loan . brach_name, loan.amount, borrower. Customer_name loan
right outer join borrower on loan.loan_number = borrower. Loan_number;

Loan_no.	Branch_name	amount	Loan_no.	Customer_name
L-170	KATHMANDU	3000	L-170	JONE
L-230	BHAKTAPUR	4000	L-230	SMITH
L-155	null	null	null	MICLE

Remarks: Tuples from the right hand-side that do not match any Tuple in the left hand side relation are padded with nulls.

SELECT loan. loan_number, loan. branch_name, loan. amount, borrower customer_name loan
Full outer join burrower on loan.loan_number = borrower. Loan_number;

loan number	branch name	amount	customer name
L - 170	Kathmandu	3000	JONES
L - 230	Bhaktapur	4000	SMITH
L - 260	Lalitpur	1700	NULL
L -150	NULL	NULL	MICHALE

Self join

- In some situation, we may need necessary to join a table to itself as we are joining two separate tables, this is known as self-join
- In a self join two rows from the same table combine to form a result row.
- Example: Consider a relation `employee' as below.

Empno	**name**	**manager no**
E001	Smith	E002
E002	Michale	E005
E003	John	E004
E004	Ivan	
E005	Scott	

Retrieve the names of employees and the names of their respective manager from the employee relation.

```
SELECT emp name , mgr name   "Manager"
      FROM  employee  emp,  employee mgr
            WHERE emp. Manager no = mgr. emp_no;
```

Output:

Name	Manager
Smith	Michael
Michael	Scott
John	Ivan

Process:

EMP

Emp no	name	manager no
E001	Smith	E002
E002	Michael	E005
E003	John	E04

MGR

emp no	name	manager no
E001	Michael	E002
E002	Scott	E005
E003	Ivan	E004

Name	Manager
Smith	Michael
Michael	Scott
John	Ivan

Data definition Language in SQL

- In SQL, DDL specifies set of relations (tables) in a database.
- SQL DDL also allows to specify
  - Integrity constraints.
  - Index on relations
  - Security and authorization for each relation
  - Physical storage structure of each relation

- Basic statement in Data Definition Language are CREATE, DROP, ALTER

Some Domain types in SQL (Data type in SQL)

CHAR(n): fixed length character string with user specified length n.

VARCHAR2(n) : A variable length character string with user specified length n. Full form is character varying and to indicate version of the domain.

NUMBER(n): holds fixed number specified length n.

NUMBER(P,S): holds fixed or floating point numbers P determines the maximum length of data, and S Determines the number of places to the right of decimal. If S is not specified then default is zero ; in such case or specified 0, it can not hold floating point number.

INT : An integer , small int

FLOAT(n) : A floating point number , with precision of at least n digits.

DATE : Represents date and time. The standard format is DD-MM-YY

LONG: Used to store variable length character strings containing up to 2 GB.

RAN/ LONG RAW : Used to store binary data such as digitized picture or image. It can contain up to 2 GB.

Schema Definition in SQL

- CREATE TABLE command is usedto create relation

Syntax :

```
CREATE TABLE <relation name>
(       A1D1 , A2D2, ..... , AnDn ,
        [< integrity constraint 1>]
        [< integrity constraint K>]
);
```

- Here, Ai is the name of attributes.
- Di is the domain type (or data type)

And integrity constraint includes:

PRIMARY KEY :

Primary key is an attribute or combination of multiple attributes that uniquely identifies records.

If a primary key is a combination of multiple attributes called composite primary key. A primary key attributes are required NOT NULL AND UNIQUE. That is primary key attribute cannot be left null and it cannot contain duplicate values.

NOT NULL / UNIQUE: Attribute can be specified NOT NULL attribute or unique attribute.

FOREIGN KEY: Any column (attribute) of table (relation) can be specified as a foreign key if it is a common attribute between relations where we are going to establish a relationship.

- In one relation (master table) it should be primary key and in another table (detail table) some attribute should be foreign key.
- Primary key and foreign key together used to establish the relationship between the two relations.
- The concept of primary key and foreign key is very important in RDBMS.

CHECK(P) : Check clause specifies the predicate P that must satisfy specified condition.

Example: SQL data definition for the simple banking database.

```
CREATE TABLE customer
        Customer_ name VARCHAR2(20) NOT NULL,
        Customer location VARCHAR2 (20)
        Constraint PK_Cname Primary key (Customer name));

CREATE TABLE branch
 (
   branch_name VARCHAR 2(15),
   branch_city VARCHAR2(30) DEFAULT " KATHMANDU" ,
   assets NUMBER (5) ,
   CONSTRAINT PK_branch_name PRIMARY KEY (branch_name),
   CONSTRAINT ch_accbal CHECK (balance >=0)
);
```

```
CREATE TABLE depositor
 (
  customer_name  VARCHAR2(20),
  account no  CHAR(10)
  CONSTRAINT fk_depositor_cname
         FOREIGN KEY(customer_name)  REFERENCES  customer,
  CONSTRAINT PK_cname_accno PRIMARY KEY (custome_name, account no)
 );

CREATE TABLE loan
 (
   loan_no CHAR(10) PRIMARY KEY ,
   branch_name  VARCHAR2(15) NOT NULL,
   amount NUMBER (5),
   CONSTRAINT fk_loan_branch_name
              FOREIGN KEY  (branch_name) REFERENCES branch,
   CHECK (amount>=0)
 );
CREATE  TABLE borrower
(
 customer_name    VARCHAR2(20),
 loan_no             CHAR(10),
 CONSTRAINT  fk_ cname  FOREIGN KEY (customer_name) REFERENCES customer
);
.
```

Example of using UNIQUE key and DEFAULT value

```
 e.g.  CREATE TABLE student
       (
         student_id  NUMBER (3) PRIMARY KEY,
         name        CHAR  (20)  UNIQUE,
         degree      CHAR (15)   DEFAULT ` Master' ,
         CHECK   (degree IN ( `Bachelors' , `Master' , `Doctorate'))
       );
```

Drop statement

- Drop table statement used to drop the relation.

Syntax:

```
DROP TABLE  <relation name>;
```

DROP user statement

- DROP USER statement is used to drop the user.

Syntax:

```
DROP USER  <user name>  [USER CASCADE];
```

ALTER TABLE statement

- Alter Table command is used to add or modify attributes to the existing relation.

Syntax:

```
ALTER TABLE <relation> ADD (attribute domain type);
```

```
ALTER TABLE <relation> MODIFY (attribute domain type);
ALTER TABLE <relation>  DROP CONSTRAINT  <constraint_name>;
ALTER TABLE <relation>  DROP  COLOUMN  <coloumn_name>;
```

Examples:

```
ALTER TABLE customer   ADD PRIMARY KEY (customer_name);
ALTER TABLE customer   ADD (customer_adds  VARCHAR2(23));
ALTER TABLE customer   MODIFY (customer_adds  VARCHAR2(32) NOT NULL);
ALTER TABLE customer  DROP PRIMARY KEY;
ALTER TABLE customer   DROP CONSTRAINT fk_cname;
ALTER TABLE customer  DROP COLOUMN customer_adds
```

View

- View is a virtual table, it does not contain actual data it is map to the base table/s.
- When tables are created or populated with data, we may require to prevent all user from accessing all columns of table. So for the data security reasons view are created.
- One alternative solution is to create several table having appropriate no of columns and assigned each user to each table. This provides well data security but it keeps redundant data in tables. So it is not useful practically. So, views are generally created instead of it . It reduces redundant data.
- View can be created from a single table hiding some coloumn/s or from the multiple tables mapping all or some of the columns of the base tables. View is the simple and effective way of hiding columns of tables for security reason.
- When view is referenced then only it holds data, so it reduces redundant data.
- When view is used to manipulate table , the underlying base e table/s are completely invisible. This adds level of data security.
- Since view can be created from multiple tables so it makes easy to query multiple tables because we can simply query views instead of query multiple tables.
- View may be read only or updateable view. Read only view only allow to read data from view. Updatable view allow insert, update and delete on view.
- If view is created from multiple tables it won't be updateable.
- If view is created without primary key and null columns then value/record can be inserted in view.
- The general syntax for view is

```
CREATE VIEW <view name>  AS  < query expression> ;
```

Example: creating view from single table.

```
CREATE VIEW   vw_emp   AS
      SELECT empno,  ename, job FROM emp;
  View coloumn can be renamed as below:
  CREATE VIEW vw_emp  AS
      SELECT empno "Employee no", ename " Employee name"
                                    Job "work" FROM  emp;
```

- Creating view from multiple tables.

```
CREATE VIEW vw_emp_info AS
      SELECT e.empno, e.ename, e.ejob, d.dname
            FROM emp e, dept d
      WHERE e.empno = d.dept no AND  e.sal>1000;
```

Common restrictions on view

- We cannot use delete statement on multiple table view.
- We cannot use insert statement unless all NOT NULL columns On underlying table are included.
- View must be created from single table to allow insert or update on view.

- If we use DISTINCT clause to create views, we cannot update or insert records within that view.

Common application of views

- Provides user security functions.
- Simplifies the constructions of complex queries.
- Summarize data from multiple tables.

Common restrictions on updatable views

- For the views to be updateable, the view definitions must not include.
- Aggregate function.
- DISTINCT, GROUP BY or HAVING clause.
- Sub – queries.
- Constraints, strin or value expression like bal*1.05
- UNION, INTERSECT, OR MINUS/EXCEPT clause.
- View can be destroyed by using DROP VIEW command.

Syntax:

DROP VIEW <view name>;
e.g. DROP view vw_emp;

Transactions:

- A transaction consists of sequence of query / or updateable statements.
- SQL standard specifies, the transaction begins implicitly when SQL statement is executed and one of the following SQL statement must end with transaction commands.

COMMIT:

It commits (save) the current transaction changes on database / table/s by update statements. After the transaction is committed, a new transaction is automatically started.

ROLLBACK:

It rollback (undo) the current transaction. That is, it undo all the update performed by SQL statements. Thus database state is restored to what it was before the first statements of the transaction were executed.

- If program terminates without executing either of the commands commit or rollback. The updates or changes to database are either committed or rollback. This depends upon SQL implementation.
- In many SQL implementations, if transactions are continued and at the same moment if the system is restarted or fails then transaction is rollback.

SQL ALIAS - AS KEYWORD

Alias is used to give an alias name to a table or a column, which can be a result set table too. This is quite useful in case of large or complex queries. Alias is mainly used for giving a short alias name for a column or a table with complex names.

Syntax of Alias for table names,

```
SELECT column-name
FROM table-name AS alias-name;
```

Example 6.56: Let's take a department table as below,

Department		
Dept_id	Dept_name	Dept_block_no
1	Computer	100
2	Mathematics	200
3	Economics	300
4	Account	400
5	Physics	500

Suppose we display above table by changing their attribute name as below,

```
SELECT Dept_id AS id, Dept_name AS name, Dept_block_no AS DNO
FROM Department;
```

By executing above query statement we get following output,

ID	NAME	DNO
1	Computer	100
2	Mathematics	200
3	Economics	300
4	Account	400
5	Physics	500

NESTED QUERY IN SQL (SUB QUERY)

A sub query is a SELECT query that is contained inside another query. The inner SELECT query is usually used to determine the results of the outer select query. A Subquery or Inner query or a Nested query is a query within another SQL query and embedded within the WHERE clause. A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved. Sub-queries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc. There are a few rules that sub-queries must follow:

- Sub-queries must be enclosed within parentheses.
- A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
- An ORDER BY command cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY command can be used to perform the same function as the ORDER BY in a subquery.
- Sub-queries that return more than one row can only be used with multiple value operators such as the IN operator.
- A subquery cannot be immediately enclosed in a set function.

Advantages of sub-queries

These are the main advantages of sub-queries:

- Provide an alternative way to query data that would require complex joins and unions.
- Make the complex queries more readable.
- Allow a complex query to be structured in a way that it is possible to isolate each part.

Sub-queries can appear in various parts of a query, including the SELECT clause, the FROM clause, the WHERE clause and the HAVING clause.

Types of Sub-queries

1. Single Row Sub Query
2. Multiple row sub query
3. Correlated Sub Query

Single Row Sub Query

Sub-query which returns single row output. They mark the usage of single row comparison operators, when used in WHERE conditions. A single-row subquery is used when the outer query's results are based on a single, unknown value. Although this query type is formally called "single-row," the name implies that the query returns multiple columns-but only one row of results. However, a single-row subquery can return only one row of results consisting of only one column to the outer query.

Syntax,

```
SELECT  field_name
FROM    table_name
WHERE   field_name Comparison operator (    SELECT  field_name
                                            FROM table_name
                                            WHERE Inner_Condition;
                                        );
```

Example 6.57: In the below SELECT query, inner query returns only one row i.e. the minimum department id for the student. It, in turn, uses this value to compare the department id of all the Students and displays only those, whose department id is equal to minimum department id.

```
SELECT Stu_name, Stu_address, Dept_id
FROM Student
WHERE Dept_id = (SELECT MIN (Dept_id)
                 FROM Student);
```

Stu_name	Stu_address	Dept_id
Maya	Palpa	1
Aarav	Ktm	1
Pinky	Ktm	1

By executing above query statements we will get following output,

Example 6.58: Find name and department id of the student of department id greater than average department id of the students.

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Dept_id > (SELECT AVG (Dept_id)
                 FROM Student);
```

Stu_name	Dept_id
Ashna	3
Anuj	4

By executing above query statements we will get following output,

Multiple row sub query

Multiple-row sub-queries are nested queries that can return more than one row of results to the parent query. Multiple-row sub-queries are used most commonly in WHERE and HAVING clauses. Since it returns multiple rows, it must be handled by set comparison operators (IN, ALL, ANY).While IN operator holds the same meaning as discussed in above, ANY operator compares a specified value to each value returned by the sub query while ALL compares a value to every value returned by a sub query.

Syntax,

```
SELECT  field_name
FROM    table_name
WHERE   field_name Relational operator ALL | ANY | SOME | IN (SELECT  field_name
                                                            FROM table_name
                                                            WHERE Inner_Condition;
                                                            );
```

Usage of Multiple Row operators

- [> ALL] More than the highest value returned by the subquery
- [< ALL] Less than the lowest value returned by the subquery
- [< ANY] Less than the highest value returned by the subquery
- [> ANY] More than the lowest value returned by the subquery
- [= ANY] Equal to any value returned by the subquery (same as IN)
- [>SOME] Same as ANY

Example 6.59: Let's take following two relations Teacher and student

Teacher

Tid	Tname	Age
1	Arjun	50
2	Choakraj	30
3	Sukraj	40
4	Prabesh	45
5	Jadish	25

Student

Sid	Sname	Tid	Age
11	Rajan	1	25
22	Pratima	2	33
33	Aarav	1	19
44	Kamala	3	35
55	Sandhya	4	30
23	Ramu	5	42

Example 6.59.1: Find id, name, and age of all teachers whose age is greater than age of all students.

```
SELECT Tid, Tname, Age
FROM Teacher
WHERE Age >ALL (SELECT Age FROM Student);
```

Tid	Tname	Age
1	Arjun	50
4	Prabesh	45

By executing above query statements we will get following output,

Example 6.59.2: Find id, name and Age of all students whose age is greater than at least one teacher.

```
SELECT Sid, Sname, Age
FROM Student
WHERE Age > SOME (SELECT Age FROM Teacher)
```

By executing above query statements we will get following output,

Sid	Sname	Age
23	Ramu	42
44	Kamala	35
22	Pratima	33
55	Sandhya	30

Example 6.59.3: Find id, name, and age of all teachers whose age is less than age of any students.

```
SELECT Tid, Tname, Age
FRO M Teacher
WHERE Age <ANY (SELECT Age FROM Student);
```

Tid	Tname	Age
5	Jadish	25
2	Choakraj	30
3	Sukraj	40

Correlated Sub Query

Correlated sub-queries depend on data provided by the outer query. This type of subquery also includes sub-queries that use the EXISTS, NOT EXISTS operators to test the existence of data rows satisfying specified criteria.

The EXISTS condition is used in combination with a subquery and is considered to be met, if the subquery returns at least one row. It can be used in a SELECT, INSERT, UPDATE, or DELETE statement. The EXISTS operator tests for the existence of rows in the results set of the subquery. If a subquery row value is found, EXISTS subquery is TRUE and in this case NOT EXISTS subquery is FALSE.

Syntax:

```
SELECT column1
FROM table1
WHERE EXISTS (SELECT * FROM table2);
```

Example 6.60: Using EXISTS Condition with the SELECT Statement

Customer

Cid	Cname	Address
C4	Aarav	Ktm
C5	Bipin	Pokhara
C6	Manmohan	Ktm
C7	Rajan	Palpa
C8	Pappu	Ktm
C9	Abin	Pokhara

Order

Oid	Cid	Odate
1	C7	2016/04/18
2	C5	2016/04/18
3	C8	2016/04/19
4	C4	2016/04/20

Now let's find all of the records from the **customers** table where there is at least one record in the orders table with the same customer id.

```
SELECT *
FROM Customer
WHERE EXISTS (
        SELECT *
        FROM Order
        WHERE Customer.Cid = Order.Cid
    );
```

Cid	Cname	Address
C4	Aarav	Ktm
C5	Bipin	Pokhara
C7	Rajan	Palpa
C8	Pappu	Ktm

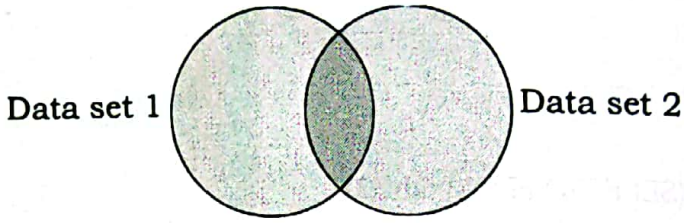
SET OPERATIONS IN SQL

The set operators are used to combine the results of two component queries into a single result. There are situations when we need to combine the results from two or more SELECT statements. SQL enables us to handle these requirements by using set operations. The result of each SELECT statement can be treated as a set, and SQL set operations can be applied on those sets to arrive at a final result. Oracle SQL supports the following four set operations:

- UNION
- UNION ALL
- INTERSECT
- MINUS

UNION Operation

UNION is used to combine the results of two or more SELECT statements. However it will eliminate duplicate rows from its result set. In case of union, number of columns and data type must be same in both the tables, on which UNION operation is being applied.



Syntax,

```
SELECT expression1, expression2, ..., expression_n
FROM tables
[WHERE conditions]
UNION
SELECT expression1, expression2, ..., expression_n
FROM tables
[WHERE conditions];
```

Example 6.61: Let's take following Supplier and order tables,

Supplier

supplier_id	supplier_name
1000	Microsoft
2000	Oracle
3000	Apple
4000	Samsung

Order

order_id	order_date	supplier_id
1	2015-08-01	2000
2	2015-08-01	6000
3	2015-08-02	7000
4	2015-08-03	8000

```
SELECT supplier_id
FROM supplier
UNION
SELECT supplier_id
FROM order
ORDER BY supplier_id;
```

We will get following output,

supplier_id
1000
2000
3000
4000
6000
7000
8000

In this SQL UNION operator example, if a supplier_id appeared in both the supplier and order table, it would appear once in result set. The UNION operator removes duplicates. If you do not wish to remove duplicates, try using the UNION ALL operator as below.

UNION ALL

UNION ALL operator is used to combine the result sets of two or more SELECT statements. It does not remove duplicate rows between the various SELECT statements (all rows are returned). Each SELECT statement within the UNION ALL must have the same number of fields in the result sets with similar data types.

Syntax

```
SELECT expression1, expression2 ..., expression_n
FROM tables
[WHERE conditions]
UNION ALL
SELECT expression1, expression2 ..., expression_n
FROM tables
[WHERE conditions];
```

Example 6.62:

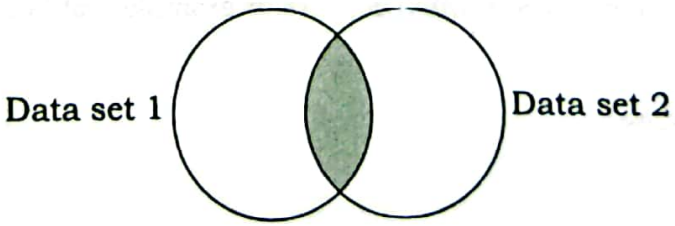
```
SELECT supplier_id
FROM supplier
UNION ALL
SELECT supplier_id
FROM order
ORDER BY supplier_id;
```

We will get the following results:

supplier_id
1000
2000
2000
3000
4000
6000
7000
8000

INTERSECT

Intersect operation is used to combine two SELECT statements, but it only returns the records which are common from both SELECT statements. In case of Intersect the number of columns and data type must be same.



Syntax,

```
SELECT expression1, expression2 ..., expression_n
FROM tables
[WHERE conditions]
INTERSECT
SELECT expression1, expression2 ..., expression_n
FROM tables
[WHERE conditions];
```

Example 6.63: Let's find intersection of above Supplier and Order tables

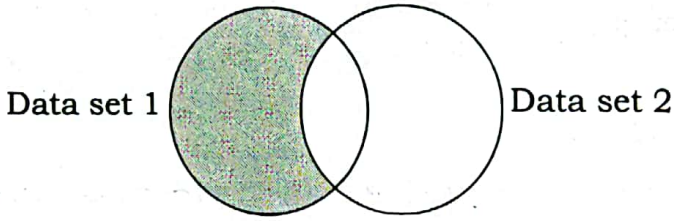
```
SELECT supplier_id
FROM supplier
INTERSECT
SELECT supplier_id
FROM order;
```

supplier_id
2000

By executing above statement we will get following result,

MINUS Set Operation

The SQL MINUS operator is used to return all rows in the first SELECT statement that are not returned by the second SELECT statement. Each SELECT statement will define a dataset. The MINUS operator will retrieve all records from the first dataset and then remove from the results all records from the second dataset.



Syntax

```
SELECT expression1, expression2 ..., expression_n
FROM tables
[WHERE conditions]
MINUS
SELECT expression1, expression2 ..., expression_n
FROM tables
[WHERE conditions];
```

Example 6.64: The following is a SQL MINUS operator example that has one field with the same data type:

```
SELECT supplier_id
FROM supplier
MINUS
SELECT supplier_id
FROM order;
```

supplier_id
1000
3000
4000

By executing this we get following output,

DCL (DATA CONTROL LANGUAGE)

DCL commands are used to assign security levels in database, which involves multiple user setups. They are used to grant defined role and access privileges to the users. To perform any operation in the database, such as for creating tables, sequences or views, a user needs privileges. Privileges are of two types,

- **System privilege**
- **Object privilege**

System privilege

Schema level privilege which is granted by DBA to the users is known as System privilege. They include privilege to issue commands like CREATE (cluster, database link, directory, job, procedure, role, synonym, table, trigger, table space, types, view, database), ALTER, DROP, DEBUG, FLASHBACK, LOCK, CONNECT, RESOURCE etc. Please note that these privileges are not limited to an object access, but applicable at User level.

Object privilege

This includes permissions for any command or query to perform any operation on the database tables. Object level privilege can be granted from object owning user (Grantor) to other user (GRANTEE). Grantor permits access on a specific object and grantee enjoys the access privilege only on the database object concerned. It may be DELETE, SELECT, INSERT, UPDATE, EXECUTE, INDEX, READ, WRITE, ALTER (table, sequence).

There are two types of DCL commands in Oracle:

1. GRANT
2. REVOKE

GRANT

A DBA or user can grant access permission on owned database objects to other user or roles using GRANT command.

Syntax:

```
GRANT [privilege]
ON [object]
TO {user |PUBLIC |role}
[WITH ADMIN | GRANT OPTION];
```

Example 6.65: Suppose U1 is a newly created user in the database. DBA shall grant below privileges to user in order to connect to the database.

```
SQL> GRANT CREATE SESSION TO U1;
```

Above privilege will enable U1 to create a session in the database.

```
SQL> GRANT CONNECT, RESOURCE TO U1;
```

Above privilege will enable U1 to connect to the database.

Example 6.66: Suppose U1 is the owner of table T1 and Procedure P1. U2 is another user in same database seeking access on T1 and P1. Grantor U1 can issue below DCL commands.

```
SQL> GRANT SELECT ON T1 TO U2;
```

Above privilege will give only SELLECT access on T1 to U2

```
SQL> GRANT EXECUTE ON P1 TO U2;
```

Above privilege will give only EXECUTE privilege on P1 to U2

ADMIN/GRANT option

ADMIN option allows the Grantee to grant a System privilege to other user. Only DBA or user with GRANT ANY PRIVILEGE system privilege can grant a system privilege to other users. Users which possess a system privilege with ADMIN option can pass only that system privilege.

Example 6.67: DBA grants CREATE INDEX privilege to user U1 with admin option

```
SQL> GRANT CREATE INDEX TO U1 WITH ADMIN OPTION
```

Now user U1 can grant CREATE INDEX system privilege to another user U2

```
SQL> GRANT CREATE INDEX TO U2
```

Note that U2 has been granted the CREATE INDEX system privilege without ADMIN OPTION. So U2 cannot pass this privilege further to its peers.

GRANT option allows the Grantor (owner) to grant an Object privilege on an object to other user. To grant an object privilege, grantor must satisfy any of the below criteria:

- He must own the object
- He must have access on object with GRANT OPTION from the owner of the object
- He must have GRANT ANY OBJECT PRIVILEGE system privilege and an object privilege on the object

Example 6.68: User U1 owns a Procedure P1. U2 seeks access privilege on P1 from U1. U1 would grant EXECUTE privilege on the table with GRANT OPTION as below

```
SQL> GRANT EXECUTE ON P1 TO U2 WITH GRANT OPTION
```

Now U2 can grant EXECUTE on P1 to other users too. In the below command, U2 grants EXECUTE ON P1 privilege to U3 but not with GRANT OPTION

```
SQL> GRANT EXECUTE ON P1 TO U3
```

REVOKE

The DCL command is used to revoke an existing privilege from a user. It can revoke a system privilege, object privilege or a role from a user. Only DBA or a user with ADMIN OPTION can revoke system privilege.

Syntax:

```
REVOKE [privilege]
ON [object]
FROM {USER |PUBLIC | ROLE}
```

Example 6.69: Below command will REVOKE a SELECT privilege on T1 table from U1.

```
SQL> REVOKE SELECT ON T1 FROM U1;
```

CREATING AND ALTERING VIEW

A view is a logical representation of another table or combination of tables. A view derives its data from the tables on which it is based. These tables are called base tables. Base tables might in turn be actual tables or might be views themselves. All operations performed on a view actually affect the base table of the view. You can use views in almost the same way as tables. You can query, update, insert into, and delete from views, just as you can standard tables. Views can provide a different representation (such as subsets or supersets) of the data that resides within other tables and views. Views are very powerful because they allow you to tailor the presentation of data to different types of users.

Advantages of Views

- Database security: view allows users to access only those sections of database that directly concerns them.
- View provides data independence.
- Easier querying
- Shielding from change
- Views provide group of users to access the data according to their criteria.
- Vies allow the same data to be seen by different users in different ways at the same time.

Creating View in SQL

We use the CREATE VIEW statement to define a view in SQL,

Syntax,

```
CREATE VIEW view_name AS
SELECT columns
FROM tables
[WHERE conditions];
```

Example 6.70: Let's take following two tables Subject and Marks,

Subject

Sub_id	Sub_name	Sub_code	Staff_id
20	DBMS	D-20	11
21	C++	C-21	22
22	NM	N-22	11
23	TOC	T-23	77
24	PHP	P-24	44
25	AI	A-25	33
26	ASP	A-26	55
27	CG	C-27	66
28	C-Prog	C-28	44

Marks

Marks_Obtain	Sub_id	Stu_id
60	20	10
55	21	12
58	22	10
49	20	13
61	23	14
67	22	13
60	24	16
55	26	15
33	25	11

Example 6.70.1: Create a view of name Subject_view with attributes subject name and subject code as,

```
CREATE VIEW Subject_view AS
SELECT Sub_name, Sub_code
FROM Subject;
```

And if we displaying above view as,

```
SELECT * FROM subject_view;
```

We get following output,

Sub_name	Sub_code
DBMS	D-20
C++	C-21
NM	N-22
TOC	T-23
PHP	P-24
AI	A-25
ASP	A-26
CG	C-27
C-Prog	C-28

Example 6.70.2: Create a view of name Subject_marks that contain only those subject name and marks obtain of the subject of marks greater than 55.

```
CREATE VIEW Subject_marks AS
SELECT Sub_name, Marks_obtain
FROM Subject NATURAL JOIN Marks
WHERE Marks_Obtain>55;
```

And if we displaying above view as,

```
SELECT * FROM subject_Marks;
```

We get following output,

Sub_name	Marks_Obtain
DBMS	60
NM	58
NM	67
TOC	61
PHP	60

Altering View

We can use the ALTER VIEW statement to explicitly recompile a view that is invalid or to modify view constraints.

In Oracle, the CREATE OR REPLACE VIEW statement is used to modify the definition of an Oracle VIEW without dropping it.

Syntax:

```
CREATE OR REPLACE VIEW view_name AS
SELECT columns
FROM table
WHERE conditions;
```

Drop VIEW

Once an Oracle VIEW has been created, you can drop it with the Oracle DROP VIEW Statement.

Syntax

```
DROP VIEW view_name;
```

Example 6.71: Here is an example of how to use the Oracle DROP VIEW Statement:

```
DROP VIEW Subject_marks;
```

This Oracle DROP VIEW example would drop/delete the Oracle VIEW called Subject_marks.

TRUNCATE TABLE

In Oracle, TRUNCATE TABLE statement is used to remove all records from a table. It works same as DELETE statement but without specifying a WHERE clause. It is generally used when you don't have to worry about rolling back. Once a table is truncated, it can't be rolled back. The TRUNCATE TABLE statement does not affect any of the table's indexes, triggers or dependencies.

Syntax,

```
TRUNCATE TABLE [Schema_name.]table_name
```

The comparisons between TRUNCATE and DELETE is that both will remove the data from the table but the main difference is that we can roll back the DELETE statement whereas we can't roll back the TRUNCATE TABLE statement.

DQL (DATA QUERY LANGUAGE)

DQL commands are basically SELECT statements. SELECT statements let you query the database to find information in one or more tables, and return the query as a result set. It is used to fetch the information from the database which is already stored there. Simply the SQL commands used to retrieve the data from database is known as Data Query Language (DQL). One such SQL command in Data Query Language used is SELECT statement which is used as information retrieval from the database.

Syntax,

```
SELECT [ALL | DISTINCT] fieldnames
FROM Table_name
[WHERE Conditions];
```

By default ALL is used in the SELECT statement.

Where, SELECT statement is used to list the required columns from given table. FROM statement is used to list the tables and without FROM statement the SELECT statement cannot display any fields of tables. And WHERE statement is optional which is used to test the condition within given columns listed by SELECT statement.

SELECT statement can be done one of the following ways:

1. Query data from a single column
2. Querying data from multiple columns
3. Querying data from all columns of a table

Example 6.4: Let's take following relational database schema for the college management system:

Department (Dept_id, Dept_name, Dept_block_no)
Student (Stu_id, Stu_name, Stu_address, Dept_id, Age)
Staff (Staff_id, staff_name, Dept_id)
Subject (Sub_id, Sub_name,Sub_code, staff_id)
Marks (Marks_Obtain, Sub_id, Stu_id)

The tables with different data for above database schema are as below;

Department

Dept_id	Dept_name	Dept_block_no
1	Computer	100
2	Mathematics	200
3	Economics	300
4	Account	400
5	Physics	500

Staff

Staff_id	Staff_name	Dept_id
11	Mohan	1
22	Pratima	2
33	Madan	1
44	Kamala	3
55	Sandhya	4
66	Umesh	3
77	Ramesh	1

Student

Stu_id	Stu_name	Stu_address	Dept_id
10	Maya	Palpa	1
11	Abin	Ktm	2
12	Aarav	Ktm	1
13	Ashna	Palpa	3
14	Anuj	Pokhara	4
15	Manish	Banepa	2
16	Pinky	Ktm	1

Marks

Marks_Obtain	Sub_id	Stu_id
60	20	10
55	21	12
58	22	10
49	20	13
61	23	14
67	22	13
60	24	16
55	26	15
33	25	11

Subject

Sub_id	Sub_name	Sub_code	Staff_id
20	DBMS	D-20	11
21	C++	C-21	22
22	NM	N-22	11
23	TOC	T-23	77
24	PHP	P-24	44
25	AI	A-25	33
26	ASP	A-26	55
27	CG	C-27	66
28	C-Prog	C-28	44

Query data from a single column

Here SELECT statement is used to retrieve a single column from given table.

Syntax,

```
SELECT column_name
FROM Table_name
[WHERE Conditions];
```

Example 6.5: suppose we need to display all department names then it can be done in SQL as,

```
SELECT Dept_name
FROM Department;
```

By executing above query we get following result

Dept_name
Computer
Mathematics
Economics
Account
Physics

Example 6.6: Find name of all departments of block number greater than 300

```
SELECT Dept_name
FROM Department
WHERE Dept_block_no >300;
```

By executing above query we get following result

Dept_name
Account
Physics

Querying data from multiple columns

To query data from multiple columns, we specify a list of comma-separated column names.

Syntax,

```
SELECT column_name1, column_name2…….. column_namen
FROM Table_name
[WHERE Conditions];
```

Example 6.7: Find name and address of all students.

```
SELECT Stu_name, Stu_address
FROM Student;
```

By executing above query we get following result

Stu_name	Stu_address
Maya	Palpa
Abin	Ktm
Aarav	Ktm
Ashna	Palpa
Anuj	Pokhara
Manish	Banepa
Pinky	Ktm

Example 6.8: Find name and address and Department number of all students of department identification number equal to 1.

```
SELECT Stu_name, Stu_address, Dept_id
FROM Student
WHERE Dept_id=1;
```

By executing above query we get following result

Stu_name	Stu_address	Dept_id
Maya	Palpa	1
Aarav	Ktm	1
Pinky	Ktm	1

Querying data from all columns of a table

Here we can use the shorthand asterisk (*) to instruct Oracle to return data from all columns of a table.

Syntax,

```
SELECT *
FROM Table_name
[WHERE Conditions];
```

Example 6.9: Find all records of Department relation

```
SELECT Dept_id, Dept_name, Dept_block_no
FROM Department;
```

Equivalently,

```
SELECT *
FROM Department;
```

By executing above query we get following result

Dept_id	Dept_name	Dept_block_no
1	Computer	100
2	Mathematics	200
3	Economics	300
4	Account	400
5	Physics	500

Example 6.10: Find all records of Department relation of department id greater than 3.

```
SELECT *
FROM Department
WHERE Dept_id>3;
```

By executing above query we get following result

Dept_id	Dept_name	Dept_block_no
4	Account	400
5	Physics	500

SQL SELECT DISTINCT Statement

The SQL SELECT DISTINCT Statement is used to retrieve unique records (by removing the duplicates) from the specified Column in the SELECT Statement. In a table, some of the columns may contain duplicate values. This is not a problem; however, sometimes you will want to list only the different (distinct) values in a table. The DISTINCT keyword can be used to return only distinct values.

Syntax

```
SELECT DISTINCT <Column names>
FROM tables
WHERE Conditions;
```

Example 6.11: Display distinct address of the students,

```
SELECT DISTINCT Stu_address
FROM Student;
```

By executing above query we get following result

Stu_address
Pokhara
Ktm
Banepa
Palpa

FROM Clause

FROM clause is a mandatory clause in SELECT expression. It specifies the tables from which data is to be retrieved.

Syntax

```
FROM table_name...
Expressions...
```

The WHERE clause

The WHERE clause is optional. When specified, it always follows the FROM clause. The WHERE clause filters rows from the FROM clause tables. Omitting the WHERE clause specifies that all rows are used. The = (equals) comparison operator compares two values for equality. Additional comparison operators and other operators used in WHERE clauses are listed below:

WHERE Clause Operators

Operator	Description
=	Equality
<>, !=	Non-equality
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
!<	Not less than
!>	Not greater than
AND	Logical AND for predicates
OR	Logical OR for predicates
BETWEEN	Between two specified values
NOT BETWEEN	Not between two specified values
IN	Used for multiple OR, Matches a value in a list
NOT IN	Matches a value outside a list
IS NULL	Is a NULL value
IS NOT NULL	Non-NULL value
LIKE	Used for pattern matching
REGEXP_LIKE	Pattern matching with regular expressions
EXISTS	Condition is met if subquery returns at least one row

Use of various comparison operators

Equality Operator

In Oracle/PLSQL, we can use the = operator to test for equality in a query.

Example 6.12: Find name and department id of students whose address is "Ktm".

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Stu_address='Ktm';
```

By executing above query we get following result

Stu_name	Dept_id
Abin	2
Aarav	1
Pinky	1

Inequality Operator

In SQL, we can use the <> or != operators to test for inequality in a query. For example, we could test for inequality using the <> operator, as follows:

Example 6.13: Find name and address of students whose address not equal 'Ktm'.

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Stu_address<>'Ktm';
```

Stu_name	Stu_address
Maya	Palpa
Ashna	Palpa
Anuj	Pokhara
Manish	Banepa

By executing above query we get following result

Greater Than Operator

We can use the > operator in Oracle to test for an expression greater than.

Example 6.14: Find name and department id of students whose department id greater than 2.

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Dept_id>2;
```

Stu_name	Dept_id
Ashna	3
Anuj	4

By executing above query we get following result

Greater Than or equal to Operator

We can use the >= operator in Oracle to test for an expression greater than or equal to.

Example 6.15: Find name and department id of students whose department id greater than or equal to 2.

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Dept_id>=2;
```

Stu_name	Dept_id
Abin	2
Ashna	3
Anuj	4
Manish	2

By executing above query we get following result

Less than Operator

We can use the < operator in Oracle to test for an expression less than.

Example 6.16: Find name and department id of students whose department id less than 2.

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Dept_id<2;
```

Stu_name	Dept_id
Maya	1
Aarav	1
Pinky	1

By executing above query we get following result

Less than or equal to Operator

We can use the <= operator in Oracle to test for an expression less than or equal to.

Example 6.17: Find name and department id of students whose department id less than 2.

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Dept_id<=2;
```

Stu_name	Dept_id
Maya	1
Abin	2
Aarav	1
Manish	2
Pinky	1

By executing above query we get following result

Advanced Operators

In WHERE clause we can also use some additional advanced operators like IN, IS NULL, LIKE, EXISTS, OR, AND, BETWEEN etc. these are described as below,

Logical Operators

Following table shows the Logical operators supported by PL/SQL. All these operators work on Boolean operands and produce Boolean results. Let us assume variable A holds true and variable B holds false, then

Operator	Description	Examples
AND	Called the logical AND operator. If both the operands are true then condition becomes true.	(A and B) is false.
OR	Called the logical OR Operator. If any of the two operands is true then condition becomes true.	(A or B) is true.
NOT	Called the logical NOT Operator. Used to reverse the logical state of its operand. If a condition is true then Logical NOT operator will make it false.	not (A and B) is true.

Example 6.18: Find name and department id of students whose address is "Ktm" and of department id not equal to 1.

```
SELECT Stu_name, Dept_id
FROM Student
WHERE Stu_address="Ktm" AND Dept_id <>1;
```

Stu_name	Dept_id
Abin	2

By executing above query we get following result

Example 6.19: Find name of the students whose student id less than 10 or of department id not equal to 1.

```
SELECT Stu_name
FROM Student
WHERE Stu_id < 10 OR Dept_id <> 1;
```

By executing above query we get following result

Stu_name
Abin
Ashna
Anuj
Manish

BETWEEN / NOT BETWEEN operators

The SQL BETWEEN operator is used along with WHERE clause for providing a range of values. The values can be the numeric value, text value, and date. The values are defined as part of the BETWEEN range are inclusive i.e. the values that are mentioned in the range are included at the start and end values. BETWEEN operator is used to get the values from given range in select, insert, delete or update statement

Syntax of BETWEEN operators

```
SELECT Columns
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

The SQL NOT BETWEEN operator is used for getting the values as part of result set which is outside of the range specified by the BETWEEN operator.

Syntax of NOT BETWEEN operator

```
SELECT Columns
FROM table_name
WHERE column_name NOT BETWEEN value1 AND value2;
```

Example 6.20: Find id, name and department id of the students whose department id is between 1 and 2.

```
SELECT Stu_id, Stu_name, Dept_id
FROM Student
WHERE Dept_id BETWEEN 1 AND 2;
```

Stu_id	Stu_name	Dept_id
10	Maya	1
11	Abin	2
12	Aarav	1
15	Manish	2
16	Pinky	1

By executing above query we get following result

Example 6.21: Find id, name and department id of the students whose department id is not between 1 and 2.

```
SELECT Stu_id, Stu_name, Dept_id
FROM Student
WHERE Dept_id NOT BETWEEN 1 AND 2;
```

Stu_id	Stu_name	Dept_id
13	Ashna	3
14	Anuj	4

By executing above query we get following result

IN operator

The IN operator checks a value within a set of values separated by commas and retrieve the rows from the table which are matching. The IN returns 1 when the search value present within the range otherwise returns 0. It is used in place of multiple OR. If we need to test multiple conditions with a lot of OR then in such a case we can user IN operator. It is used with SELECT, INSERT, UPDATE, or DELETE statement to decrease the use of multiple OR condition.

Syntax,

```
SELECT Columns
FROM table_name
WHERE column IN (value1, value2 ... valueN);
```

Example 6.22: Find name and block number of departments of name is 'Computer' or 'Account' or 'Physics' or 'English'.

```
SELECT Dept_name, Dept_block_no
FROM Department
WHERE Dept_name IN ('Computer','Account','Physics','English');
```

By executing above query we get following result

Dept_name	Dept_block_no
Computer	100
Account	400
Physics	500

Example 6.23: Find name and block number of departments of id not in (1, 2, 3, 8, 9)

```
SELECT Dept_name, Dept_block_no
FROM Department
WHERE Dept_id NOT IN (1, 2, 3, 8, 9);
```

Dept_name	Dept_block_no
Account	400
Physics	500

By executing above query we get following result

SQL Wildcards (LIKE Operator)

Pattern matching is one of the important operations with strings. SQL allows pattern matching with strings by using LIKE operator. It matches a string value against a pattern string containing wild-card characters. The wild-card characters for LIKE are percent -- '%' and underscore -- '_'. Underscore matches any single character. Percent matches zero or more characters.

Syntax

```
SELECT column-names
FROM table-name
WHERE column-name LIKE Pattern;
```

With SQL wildcards, we can create search patterns that can be compared against given data. The wildcards themselves are actually characters that have special meanings within the SQL WHERE clauses.

Wildcard Operators

Operator	Matches
%	A percent sign matches any string of zero or more characters.
_	An underscore matches any one character.

Examples of % and _ Patterns

Pattern	Matches
'A%'	Matches a string of length ≥ 1 that begins with A, including the single letter A. E.g. Matches 'A', 'Anonymous', 'AC/DC' etc.
'%s'	Matches a string of length ≥ 1 that ends with s, including the single letter s. A string with trailing spaces (after the s) won't match. E.g. Matches strings 's', 'Victoria Falls', 'DBMSs' etc.
'%in%'	Matches a string of length ≥ 2 that contains 'in' anywhere. E.g. Matches 'in', 'inch', 'Pine', 'linchpin', 'lynchpin' etc.
'____'	Matches any four-character string. E.g. matches 'ABCD', 't am', 'Abin' etc.
'Qua___'	Matches any five-character string that begins with Qua. E.g. matches 'Quack', 'Quaff', 'Quake' etc.
'_re_'	Matches any four-character string that has string're' as its second and third characters. E.g. matches 'Tree', 'area', 'fret' etc.
'_re%'	Matches a string of length ≥ 3 that begins with any character and has're' as its second and third characters. E.g. matches 'Tree', 'area', 'fret', 'are', 'fretful' etc.
'%re_'	Matches a string of length ≥ 3 that has 're' as the second and third characters from its end and ends with any character. E.g. matches 'Tree', 'area', 'fret', 'red', 'Blood red' etc.

'200%'	Finds any values that start with 200
'%200%'	Finds any values that have 200 in any position
'_00%'	Finds any values that have 00 in the second and third positions.
'2_%_%'	Finds any values that start with 2 and are at least 3 characters in length
'_2%3'	Finds any values that have a 2 in the second position and end with a 3.
'2_ _ _3'	Finds any values in a five-digit number that start with 2 and end with 3.

Example 6.24: Find name, address and department id of the students whose address beginning with letter P and of department id greater than 2.

```
SELECT Stu_name, Stu_address, Dept_id
FROM Student
WHERE Stu_address LIKE 'P%'AND Dept_id>2;
```

By executing above query we get following result

Stu_name	Stu_address	Dept_id
Maya	Palpa	1
Ashna	Palpa	3

Example 6.25: Find id, name and address of the students whose address end with letter 'a' and of name of 4 letters.

```
SELECT Stu_id, Stu_name, Stu_address
FROM Student
WHERE Stu_address LIKE '%a' AND Stu_name LIKE '_ _ _ _';
```

By executing above query we get following result

Stu_id	Stu_name	Stu_address
10	Maya	Palpa
14	Anuj	Pokhara

Example 6.26: Find id, and name of all Students whose name contains characters 'a' and 'n'.

```
SELECT Stu_id, Stu_name
FROM Student
WHERE Stu_name LIKE '%a%n%' OR Stu_name LIKE '%n%a%';
```

By executing above query we get following result

Stu_id	Stu_name
13	Ashna
15	Manish

Pattern matching with regular expressions

The REGEXP_LIKE function is used for pattern matching by using regular expression. This function searches a character column for a pattern. Use this function in the WHERE clause of a query to return rows matching the regular expression you specify.

Pattern

The regular expression matching information. It can be a combination of the following:

Value	Description
^	Matches the beginning of a string. If used with a *match_parameter* of 'm', it matches the start of a line anywhere within *expression*.
$	Matches the end of a string. If used with a *match_parameter* of 'm', it matches the end of a line anywhere within *expression*.
*	Matches zero or more occurrences.
+	Matches one or more occurrences.
?	Matches zero or one occurrence.
.	Matches any character except NULL.
|	Used like an "OR" to specify more than one alternative.
[]	Used to specify a matching list where you are trying to match any one of the characters in the list.
[^]	Used to specify a non-matching list where you are trying to match any character except for the ones in the list.
()	Used to group expressions as a sub-expression.
{m}	Matches m times.
{m,}	Matches at least m times.
{m, n}	Matches at least m times, but no more than n times.
\n	n is a number between 1 and 9. Matches the nth sub-expression found within ( ) before encountering \n.
[..]	Matches one collation element that can be more than one character.
[::]	Matches character classes.
[==]	Matches equivalence classes.
\d	Matches a digit character.
\D	Matches a non-digit character.
\w	Matches a word character.
\W	Matches a non-word character.
\s	Matches a whitespace character.
\S	Matches a non-whitespace character.
\A	Matches the beginning of a string or matches at the end of a string before a newline character.
\Z	Matches at the end of a string.
*?	Matches the preceding pattern zero or more occurrences.
+?	Matches the preceding pattern one or more occurrences.
??	Matches the preceding pattern zero or one occurrence.
{n}?	Matches the preceding pattern n times.
{n,}?	Matches the preceding pattern at least n times.
{n, m}?	Matches the preceding pattern at least n times, but not more than m times.

Syntax,

```
SELECT attribute list
FROM List of tables
WHERE REGEXP_LIKE(column_name, 'regular expression');
```

Example 6.27: The following WHERE clause filters students with a name of Abin or Anuj

```
SELECT Stu_name, Stu_address
FROM Student
WHERE REGEXP_LIKE(Stu_name, '^A(bin|nuj)');
```

Stu_name	Stu_address
Abin	Ktm
Anuj	Pokhara

Example 6.28: let's use the REGEXP_LIKE condition to match on the beginning of a string.

```
SELECT Stu_name
FROM Student
WHERE REGEXP_LIKE (Stu_name, '^M(*)');
```

Stu_name
Maya
Manish

This REGEXP_LIKE example will return all students whose name starts with 'M'.

Example 6.29: Let's use the REGEXP_LIKE condition to match on the end of a string.

```
SELECT Stu_name
FROM Student
WHERE REGEXP_LIKE (Stu_name, '(*)a$');
```

Stu_name
Maya
Ashna

This REGEXP_LIKE example will return all students whose name ends with 'a'.

ORDERING THE DISPLAY OF TUPLES

SQL ORDER BY clause is used for sorting the result set. The sorting is done in either ascending or in descending order. The default order of sorting is sorting in ascending order. It is used with SQL Select queries. Sorting is done after the result set is obtained. We can use multiple columns with ORDER BY clause, sorting will happen from left side columns first and then towards the right side columns.

Syntax:

```
SELECT attribute list
FROM List of tables
[WHERE predicate]
ORDER BY column-1 [ASC|DESC] [column-2 [ASC|DESC].....];
```

Column-1, column-2 ... are column names specified, the new name is used in the ORDER BY list. ASC and DESC request ascending or descending sort for a column. ASC is the default. ORDER BY

clause sorts rows using the ordering columns in left-to-right, major-to-minor order. The rows are sorted first on the first column name in the list. If there are any duplicate values for the first column, the duplicates are sorted on the second column in the order by list, and so on. Database NULL require special processing in ORDER BY. A NULL column sorts higher than all regular values; this is reversed for DESC.

Example 6.30: Find id, name and address of all students in the order of their address in ascending order.

```
SELECT Stu_id, Stu_name, Stu_address
FROM Student
ORDER BY Stu_address ASC;
```

Here ASC is optional. Since by default order is ascending order

Stu_id	Stu_name	Stu_address
15	Manish	Banepa
11	Abin	Ktm
12	Aarav	Ktm
16	Pinky	Ktm
10	Maya	Palpa
13	Ashna	Palpa
14	Anuj	Pokhara

Example 6.31: Find id, name and department id of all students whose address is 'Ktm' in descending order of their name.

```
SELECT Stu_id, Stu_name, Dept_id
FROM Student
WHERE Stu_address='Ktm'
ORDER BY Stu_name DESC;
```

Stu_id	Stu_name	Dept_id
16	Pinky	1
11	Abin	2
12	Aarav	1

Example 6.32: Find id, name, address and department id of all students in the ascending order of their address and then in descending order of department id.

```
SELECT Stu_id, Stu_name, Stu_address, Dept_id
FROM Student
ORDER BY Stu_address, Dept_id DESC;
```

Stu_id	Stu_name	Stu_address	Dept_id
15	Manish	Banepa	2
11	Abin	Ktm	2
12	Aarav	Ktm	1
16	Pinky	Ktm	1
13	Ashna	Palpa	3
10	Maya	Palpa	1
14	Anuj	Pokhara	4

Example 6.33: Sorting by relative position

We can also use the Oracle ORDER BY clause to sort by relative position in the result set, where the first field in the result set is 1. The next field is 2, and so on.

"Find id, name and address of all students in the order of their address in ascending order by using their relative position."

```
SELECT Stu_id, Stu_name, Stu_address
FROM Student
ORDER BY 3;
```

Stu_id	Stu_name	Stu_address
15	Manish	Banepa
11	Abin	Ktm
12	Aarav	Ktm
16	Pinky	Ktm
10	Maya	Palpa
13	Ashna	Palpa
14	Anuj	Pokhara

AGGREGATE FUNCTIONS

Aggregate functions operate against a collection of values, but return a single value. It is mainly used in summarizing the information stored in tables. Aggregate functions supported by SQL are listed below:

Function	Description
AVG(column)	Returns the average value of a column
COUNT(column)	Returns the number of rows (without a NULL value) of a column
COUNT(*)	Returns the number of selected rows
MAX(column)	Returns the highest value of a column
MIN(column)	Returns the lowest value of a column
SUM(column)	Returns the total sum of a column

The result of the COUNT function is always integer. The result of all other functions is the same data type as the argument. The Aggregate Functions skip columns with nulls, summarizing non-null values. COUNT function counts rows with non-null values; AVG function averages non-null values, and so on. COUNT returns 0 when no non-null column values are found; the other functions return NULL when there are no values to summarize. The DISTINCT and ALL specifiers are optional. ALL specifies that all non-null values are summarized; it is the default. DISTINCT specifies that distinct column values are summarized; duplicate values are skipped. Note that DISTINCT has no effect on MIN and MAX results. Aggregate functions have the following general format:

Aggregate-function ([DISTINCT | ALL] column-1)

In this syntax;

- First, specify the name of an aggregate function that you want to use such as AVG, SUM, and MAX.

- Second, use DISTINCT if you want only distinct values are considered in the calculation or ALL if all values are considered in the calculation. By default, ALL is used if you don't specify any modifier.
- Third, the expression can be a column of a table or an expression that consists of multiple columns with arithmetic operators.

Example 6.34: Find minimum and maximum id of a department

```
SELECT MAX (Dept_id), MIN (Dept_id)
FROM Department;
```

MAX(Dept_id)	MIN(Dept_id)
5	1

Example 6.35: Find maximum department name.

```
SELECT MAX (Dept_name)
FROM Department;
```

MAX(Dept_name)
Physics

Example 6.36: Find average Department block number of all departments whose id less than 3.

```
SELECT AVG (Dept_block_no)
FROM Department
WHERE Dept_id<3;
```

AVG(Dept_block_no)
150

Example 6.37: Find total number of departments.

```
SELECT COUNT (*)
FROM Department;
```

COUNT(*)
5

Example 6.38: Find number of departments of name end with s.

```
SELECT COUNT (Dept_id) AS NO_Of_Department
FROM Department
WHERE Dept_name LIKE '%s';
```

No_Of_Departments
3

GROUP BY CLAUSE

The GROUP BY Statement in SQL is used to arrange identical data into groups with the help of some functions. I.e. if a particular column has same values in different rows then it will arrange these rows in a group. GROUP BY clause is used to group the results of a SELECT query based on one or more columns. It is also used with SQL functions to group the result from one or more tables. It has following properties:

- GROUP BY clause is used with the SELECT statement.
- In the query, GROUP BY clause is placed after the WHERE clause.
- In the query, GROUP BY clause is placed before ORDER BY clause if used any.

- GROUP BY aggregates the results on the basis of selected column: COUNT, MAX, MIN, SUM, AVG, etc.
- GROUP BY returns only one result per group of data.

Syntax

```
SELECT expression1, expression2, ..., expression_n, aggregate_function (aggregate_expression)
FROM tables
WHERE conditions
GROUP BY expression1, expression2, ..., expression_n;
```

Example 6.39: Find maximum department id and address of student in each address level

```
SELECT MAX(Dept_id), Stu_address
FROM Student
GROUP BY Stu_address;
```

MAX(Dept_id)	Stu_address
4	Pokhara
2	Ktm
2	Banepa
3	Palpa

Example 6.40: Count the number of students in each address level

```
SELECT COUNT(Stu_id) AS Number_Of_Students, Stu_address
FROM Student
GROUP BY Stu_address;
```

Number_Of_Students	Stu_address
1	Pokhara
3	Ktm
1	Banepa
2	Palpa

Example 6.41: Count the number of students in each address level of name start with P or K

```
SELECT count(Stu_id) as number_of_students, Stu_address
FROM Student
WHERE Stu_name LIKE 'A%'
OR  Stu_name LIKE 'P%'
GROUP BY Stu_address;
```

Number_Of_Students	Stu_address
1	Pokhara
3	Ktm
1	Palpa

HAVING CLAUSE

HAVING clause is used with SQL queries to give more precise condition for a statement. It is used to mention condition in GROUP BY based SQL queries, just like WHERE clause is used with SELECT query. Simply the GROUP BY clause is used to check the condition in each group created by GROUP BY clause.

HAVING Clause utilized in SQL as a conditional Clause with GROUP BY Clause. This conditional clause returns rows where aggregate function results matched with given conditions only. It added in the SQL because WHERE Clause cannot be combined with aggregate results, so it has a different purpose. The primary purpose of the WHERE Clause is to deal with non-aggregated or individual records. It has following major properties:

- HAVING Clause always utilized in combination with GROUP BY Clause.
- HAVING Clause restricts the data on the group records rather than individual records.
- WHERE and HAVING can be used in a single query.

Syntax

```
SELECT expression1, expression2, ...,expression_n, aggregate_function (aggregate_expression)
FROM tables
WHERE conditions
GROUP BY expression1, expression2, ...,expression_n
HAVING having_condition;
```

Example 6.42: Find maximum department id and address of student in each address level with maximum department greater than 2.

```
SELECT MAX(Dept_id), Stu_address
FROM Student
GROUP BY Stu_address
HAVING MAX(Dept_id)>2;
```

MAX(Dept_id)	Stu_address
4	Pokhara
3	Palpa

Example 6.43: Find the number of students in each address level with total students less than 3.

```
SELECT COUNT(Stu_id) AS NOS, Stu_address
FROM Student
GROUP BY Stu_address
HAVING COUNT(Stu_id)<3;
```

NOS	Stu_address
1	Pokhara
1	Banepa
2	Palpa

Example 6.44: Find maximum name and department id of students in each department level of name start with A or M.

```
SELECT MAX(Stu_name), Dept_id
FROM Student
GROUP BY Dept_id
HAVING MAX(Stu_name) LIKE 'A%' OR MAX(Stu_name)
    LIKE 'M%';
```

MAX(Stu_name)	Dept_id
Manish	2
Anuj	4
Ashna	3

Differences between WHERE and HAVING clause

- WHERE clause is employed in row operations and applied on a single row whereas HAVING clause is used in column operations and can be applied to summarized rows or groups.
- In WHERE clause the desired data is fetched according to the applied condition. In contrast, HAVING clause fetch whole data then separation is done according to the condition.
- Aggregate functions like MIN, SUM, MAX, AVG can never appear along with WHERE clause. As against, these functions can appear in HAVING clause.
- HAVING clause cannot use without a SELECT statement. Conversely, WHERE can be used with SELECT, UPDATE, DELETE, etc.
- WHERE clause behaves as a pre-filter while HAVING clause acts as a post-filter.

In tabulated form their differences are given below,

WHERE Clause	HAVING Clause
The WHERE clause specifies the criteria which individual records must meet to be selected by a query. It can be used without the GROUP BY clause.	The HAVING clause cannot be used without the GROUP BY clause.
The WHERE clause select rows before grouping.	The HAVING clause select rows after grouping.
The WHERE clause cannot contain aggregate functions	The HAVING clause can contain aggregate functions.
WHERE clause is used to impose condition on SELECT statement as well as single row function and is used before GROUP BY clause.	HAING clause is used to impose condition on GROUP function and is used after GROUP BY clause in the query.
Syntax SELECT expression1, expression2, ..., expression_n, aggregate_function (aggregate_expression) FROM tables WHERE conditions GROUP BY expression1, expression2, ..., expression_n;	**Syntax** SELECT expression1, expression2, ..., expression_n, aggregate_function (aggregate_expression) FROM tables WHERE conditions GROUP BY expression1, expression2, ..., expression_n HAVING having_condition;